ALGORITMI II : A.A. 2020-2021

I
- PARADIGMI DI SVILUPPO DI ALGORITMI
  - Programmazione Dinamica
  - Algoritmi Greedy
  - Algoritmi orientati su Grafi Pesati
    - Percorsi minimi in grafi pesati
    - Flusso su reti + Applicazioni
    - Minimo Albero di Copertura + Applicazioni

II
- PROBLEMI INTRATTABILI
  - Riduzioni Polinomiali
  - Le classi di complessità P e NP
  - Problemi NP-Completi e Intrattabilità
- Tecniche Approssimanti
- Tecniche Euristiche

} Problemi Intrattabili

LIBRI DI TESTO:

- J. KLEINBERG, É. TARDOS "ALGORITHM DESIGN"
- T. H. CORMEN et Al. "INTRODUCTION TO ALGORITHMS"

SOLUZIONE DI PROBLEMA

1) FORMALIZZAZIONE (FORMULAZIONE MATEMATICA)
   - PARAMETRI DI INGRESSO : quali sono, che forma hanno
   - SOLUZIONE : forma e proprietà

2) DEFINIZIONE PROCEDURA DI SOLUZIONE
   - Seguendo Principi di Decomposizione e/o Riduzione
   - Applicando uno dei paradigmi sopra (greedy, prog. dinamico, divide et impera, ...)

3) STABILIRE CORRETTEZZA

4) STABILIRE LIMITE SUPERIORE AL TEMPO DI ESECUZIONE
   - Procisare STRUTTURE DATI SPECIFICHE per ottenere tempi ottimali

---

PROGRAMMAZIONE DINAMICA

- Tecnica di progettazione di algoritmi
- Estende DIVIDE ET IMPERA
  - Sottoproblemi non disgiunti
  - Problemi di ottimizzazione

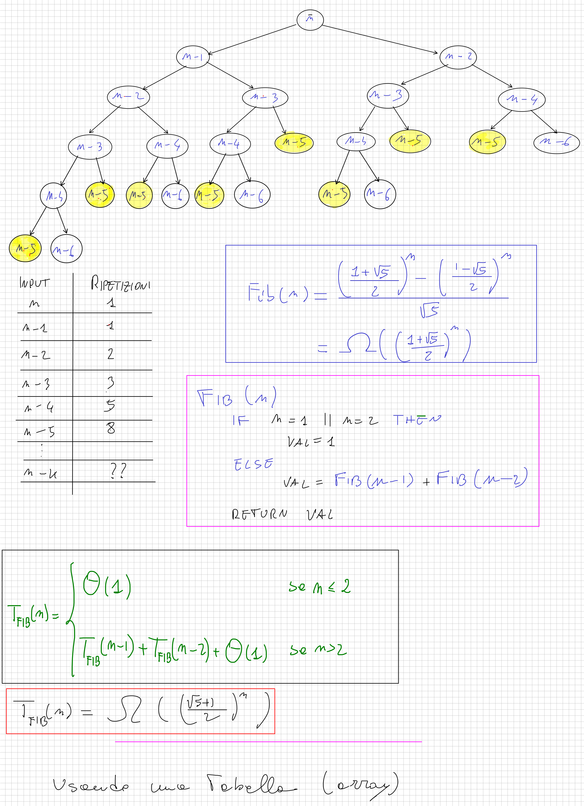
Sottoproblemi non disgiunti

- MERGESORT e QUICKSORT generano sottoproblemi disgiunti

- Funzione di FIBONACCI

$$F(n) = \begin{cases} 1 & \text{se } n = 1 \text{ o } n = 2 \\ F(n-1) + F(n-2) & \text{se } n > 2 \end{cases}$$

Albero delle chiamate ricorsive:



INPUT	RIPETIZIONI
n	1
$n-1$	1
$n-2$	2
$n-3$	3
$n-4$	5
$n-5$	8
⋮	
$n-k$	??

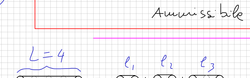
$$Fib(n) = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}} = \Omega\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$$

```
FIB (n)
  IF n = 1 || n = 2 THEN
      VAL = 1
  ELSE
      VAL = FIB(n-1) + FIB(n-2)
  RETURN VAL
```

$$T_{FIB}(n) = \begin{cases} \Theta(1) & \text{se } n \leq 2 \\ T_{FIB}(n-1) + T_{FIB}(n-2) + \Theta(1) & \text{se } n > 2 \end{cases}$$

$$T_{FIB}(n) = \Omega\left(\left(\frac{\sqrt{5}+1}{2}\right)^n\right)$$

Usando una Tabella (array)




```
FIB(n)
  IF fn < 0 THEN
      IF n = 1 || n = 2 THEN
          VAL = 1
      ELSE
          VAL = FIB(n-1) + FIB(n-2)
      fn = VAL
  RETURN fn
```

```
FIB(n)
  f1 = f2 = 1
  FOR i = 3 TO n DO
      fi = fi-1 + fi-2
  RETURN fn
```

$$T_{FIB}(n) = \Theta(n)$$

Problema: TAGLIO OTTIMO DEL TRONCO

- Un tronco d'albero di lunghezza L
- $\forall\, \ell \leq L$, p_ℓ è il prezzo di un taglio del tronco di lunghezza $\ell \leq L$

Soluzione: Qual è l'insieme di Tagli del Tronco che rende massimo il guadagno?

INSIEME di TAGLI: $S = \{\ell_1, \ell_2, \ldots, \ell_k\}$ t.c.

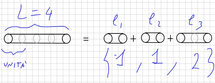
$\ell_1 + \ell_2 + \ldots + \ell_k = L$ e $C_S = \sum_{i=1}^{k} p_{\ell_i}$ è MASSIMO

$\ell_i > 0$ $\forall\, i \in \{1, \ldots, k\}$

Soluzione Ammissibile

Valore (costo) della Soluzione

ESEMPIO: $L = 4$



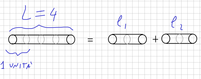
$= \ell_1 + \ell_2 + \ell_3$

$\{1, 1, 2\}$

$T_1 = 1 + 1 + 2 = 4$

$Costo(T_1) = p_{\ell_1} + p_{\ell_2} + p_{\ell_3}$

$L = 4$



$= \ell_1 + \ell_2$

$T_2 = 2 + 2 = 4$

$Costo(T_2) = p_2 + p_2$

$T_4 = 1 + 4 + 3 + 2 + 4$

$C_{STR}(T_4) = P_1 + P_4 + P_3 + P_2 + P_4$

$T_{01} = 4$

$C_{STR}(T_{01}) = P_1$

Quanti possibili modi di tagliare un tronco di lunghezza L esistono?

T_i = sequenza di $i-1$ bit; T_i sia il bit $i-esimo$

T_{i+1} = Taglio alla $i-esima$ unità di unicode

T_{i+0} = No Taglio alla $i-esima$ " " "

T_2 T_3 T_4	T
0 1 1	$\{1, 1+2\}$
1 0 0	$\{1, 3\}$
1 1 0	$\{1, 2, 2\}$
1 1 1	$\{1, 1, 1, 1\}$
0 0 0	$\{4\}$

Ogni $STRing$ di $L-1$ bit corrisponde a un possibile insieme di Tagli $\Rightarrow$ 2^{L-1} possibili soluzioni

1) STUDIO STRUTTURA SOLUZIONI

Insieme di Tagli $T = \{t_1, t_2, \dots, t_k\}$ t.c.

$t_1 + t_2 + \dots + t_k = L$ Vincolo

e $P_1 + \sum_{i=1}^{k} P_{t_i}$ è MASSIMO

Soluzione Ammissibile — Valore (costo) della Soluzione

Decomponiamo una Soluzione ammissibile per $P(L)$ in **Sottosoluzioni** per suoi Sottoproblemi

$S = \{P_1, t_2, \dots, t_k\} = \{P_1\} \cup S'$ con $S' = \{t_2, t_3, \dots, t_k\}$

dove S è soluzione del problema $P(L)$

S' è soluzione del problema $P(L - t_2)$

Teorema Se $S = \{P_1, t_2, \dots, t_k\}$ è soluzione ammissibile del prob. $P(L)$, allora $S' = \{t_2, \dots, t_k\}$ è soluzione ammissibile del problema $P(L - t_2)$

Dim.

S è ammissibile se $t_2 + t_3 + \dots + t_k = L$

$\Rightarrow$ $t_2 + \dots + t_k = L - t_2$ $\Rightarrow$ S' è ammissibile per $P(L - t_2)$

$\Rightarrow$ Ogni sol ammissibile per $P(L)$ contiene la soluzione ammissibile di un sottoproblema

(*)

Indichiamo con $S(L)$ una sol ammissibile per il problema $P(L)$

Teorema

Se $S = \{P_1, t_2, \dots, t_k\}$ è sol ammissibile per $P(L)$ e $S' = \{t_2, \dots, t_k\}$ è arbitrarie sol ammissibile per $P(L - t_2)$, allora $S^* = \{P_1\} \cup S' = \{P_1, t_2, \dots, t_k\}$ è sol. ammissibile per $P(L)$.

$\Rightarrow$ La composizione della prima scelta di una sol ammissibile per $P(L)$ e una qualsiasi sol ammissibile del sottoproblema è ancora una sol ammissibile per $P(L)$.

(*)

SOTTOSTRUTTURA OTTIMA (*)

Teorema: Se $S(L) = \{P_1, t_2, \dots, t_k\}$ è **soluzione ottima** per $P(L)$, allora $S'(L - t_1) = \{P_2, \dots, t_k\}$ è **soluzione ottima** per $P(L - t_1)$

Dim. Per assurdo. Assumiamo $S(L)$ ottima per $P(L)$ ma $S'(L - t_1)$ NON ottima per $P(L - t_1)$

.
.
.

$\Rightarrow$ Una soluzione ottima del problema $P(L)$ contiene soluzioni ottime di sottoproblemi di $P(L)$.

(*)

2) DEFINIZIONE RICORSIVA DEL VALORE (COSTO) DI UNA SOLUZIONE OTTIMA

- Una soluzione ammissibile per $P(L)$ ha la forma $S(L) = \{P_1, t_2, \dots, t_k\}$
- $S(L)$ è ottima se $C(S(L)) = P_1 + P_{t_2} + \dots + P_{t_k}$ è massimo
  - $S'(L - t_1) = \{P_{t_2} \dots t_k\}$ è ottima per $P(L - t_1)$
  - Quali sono i possibili valori per t_2? $t_1 \in \{1, 2, \dots, L\}$
  - Se $C(L)$ denota il costo di una soluzione ottima per $P(L)$, allora
  - Possiamo ottenere $C(L)$ come

$$C(L) = \max_{t_1 \in \{1, \dots, L\}} \{P_{t_1} + C(L - t_1)\}$$

[ponendo $C(0) = 0$]

In generale, il costo della soluzione ottima al problema $P(\ell)$ è espresso dalla ricorrenza

$$C(\ell) = \begin{cases} 0 & \text{se } \ell = 0 \\ \max_{t_1 \in \{1, \dots, \ell\}} \{P_{t_1} + C(\ell - t_1)\} & \text{altrimenti} \end{cases}$$

che può essere calcolata dall'algoritmo

```
TAGLIO_OTTIMO (L)
  IF L == 0 THEN
     V = 0
  ELSE
     V = -∞
     FOR i = 1 TO L DO
        V = MAX (V, P_i + TAGLIO_OTTIMO (L - i))
  RETURN V
```

Quante chiamate ricorsive esegue questo algoritmo a cascata?

$$T(L) = \begin{cases} 1 & \text{Se } L = 0 \\ 1 + \sum_{t=1}^{L} (T(L - t) + 1) & \text{altrimenti} \end{cases}$$

$$T(L) = 1 + \sum_{t=1}^{L} \left(T(L - t) + 1\right) = 1 + L + \sum_{t=1}^{L} \left(T(L - t)\right)$$

$$T(L) = 1 + L + \sum_{t=0}^{L-1} T(t) \quad \Big| \quad T(L-1) = 1 + L - 1 + \sum_{t=0}^{L-2} T(t)$$

$$T(L) = 1 + T(L-1) + L + \sum_{t=0}^{L-1} T(t) = 2 \cdot T(L-1) + 1$$

L'albero ricorsivo di questa ricorrenza è un albero binario pieno di altezza L

Il valore $T(L)$ è il n° di nodi di questo albero

$$T(L) = 2^{L+1} - 1 = \Theta(2^L)$$

Tempo è esponenziale sulla Taglia L del problema

Molte chiamate ricorsive risolvono gli stessi problemi in rami diversi dell'albero di ricorrenza dell'algoritmo

Infatti è immediato osservare che $P(L)$ ha al massimo L diversi sottoproblemi

$P(0), P(1), \dots, P(L-2), P(L-1)$

Possiamo quindi usare Tabelle (arrays) per memorizzare i valori ottimi per gli $L+1$ (sotto)problemi di $P(L)$

3) CALCOLO DEL COSTO DELLA SOLUZIONE OTTIMA

$$C(\ell) = \begin{cases} 0 & \text{se } \ell = 0 \\ \max_{i \in \{1,\dots,\ell\}} \{ p_i + C(\ell - i) \} & \text{altrimenti} \end{cases}$$

Per poter calcolare $C[\ell]$ è necessario aver già calcolato $C[i]$ per ogni $i < \ell$!!

Due Tabelle $C[\,]$ e $S[\,]$

$C[\ell]$: costo della soluzione ottima per $P(\ell)$

$S[\ell]$: prima scelta della soluzione ottima a $P(\ell)$
[se sol. ottima a $P(\ell)$ è $\{\ell_1, \ell_2, \dots, \ell_k\}$, $S[\ell] = \ell_1$]

```
TAGLIO-OTTIMO (L, P[])
  C[0] = 0
  FOR ℓ = 1 TO L DO
      v = -∞
      FOR i = 1 TO ℓ DO
          IF v < P[i] + C[ℓ - i] THEN
              v = P[i] + C[ℓ - i]
              t = i
      C[ℓ] = v
      S[ℓ] = t
```

$O(L)$

$$T_{TO}(L) = \Theta(L^2)$$

4) RICOSTRUZIONE SOLUZIONE OTTIMA

Dato l'array $S[\,]$ (scelte ottime dei sottoprob.) soluzione ottima facilmente ricostruibile.

```
PRINT_TAGLIO-OTTIMO (L, P[])
    TAGLIO_OTTIMO (L, P[])
    WHILE (L > 0) DO
        PRINT S[L]
        L = L - S[L]
```

$\Theta(L)$

$T_{PTO}(L) = \Theta(L^2)$

Stampe le scelte dei tagli che determinato il valore massimo

⇓

Soluzione Ottima

Rappresentazioni diverse delle soluzioni possono portare a
- formulazioni differenti del problema
- soluzioni algoritmiche diverse

Es.

1) Sol $\{\ell_1, \ell_2, \dots, \ell_k\}$ con ℓ_i la lunghezza dell'i-esimo taglio in ordine temporale

$\ell_i \in \{1 \dots L\} \quad \forall\, 1 \le i \le k$

$L \ge \sum_{i=1}^{k} \ell_i = L$ (ammissibilità)

$\{\ell_1, \dots, \ell_k\} = Sol(L) \Rightarrow \{\ell_2 \dots \ell_k\} = Sol(L - \ell_1)$

$$C(\ell) = \begin{cases} 0 & \text{Se } \ell = 0 \\ \max_{1 \le i \le \ell} \{ P_i + C(\ell - i) \} & \text{se } \ell > 0 \end{cases}$$

Otteniamo un n° lineare ℓ di sottoproblemi distinti, ogni scelta richiede tempo lineare su L!

2) Sol $= \langle \ell_1, \dots, \ell_m \rangle$ con ℓ_i dimensione del taglio i in ordine una decrescente di lunghezza.

$1 \le \ell_1 \;,\; \ell_k \le L$

$\ell_i \ge \ell_{i+1} \quad \forall\, 1 \le i \le k-1$

$L \ge \sum_{i=1}^{k} \ell_i = L$ (ammissibilità)

Es: $L = 10$ Sol $= \langle 1, 2, 2, 4 \rangle$

$P(i, j)$ | Problema di Tagliare un Tronco di lunghezza j in pezzi di dimensione $\ge i$ (i = unico Taglio disponibile)

Se $\langle \ell_1, \dots, \ell_m \rangle$ è $Sol(1, L)$ allora 2 casi possibili:

1) $\langle \ell_2, \dots, \ell_m \rangle$ è $Sol(2, L)$ [nessun Taglio di dimensione 1]

2) $\langle \ell_2, \dots, \ell_m \rangle$ è $Sol(1, L-1)$ [1° Taglio ha dimensione 1]

Es:

Data $Sol(1, 10) = \langle 1, 1, 2, 2, 4 \rangle \Rightarrow$

$Sol(1, 3) = \langle 1, 2, 4 \rangle$ [caso 2]

$Sol(1, 8) = \langle 2, 2, 4 \rangle = Sol(3, 8)$ (caso 2) (caso 1)

In generale:

se $\langle \ell_1, \dots, \ell_m \rangle$ è $sol(i, \ell)$
- $\langle \ell_2, \dots, \ell_m \rangle$ è $sol(i+1, \ell)$ [$\ell_1 \ne i$]
- $\langle \ell_1, \ell_2, \dots, \ell_m \rangle$ è $sol(i, \ell - i)$ [$\ell_1 = i$]

Dato $0 \le \ell \le L$ e $1 \le i \le L+1$ possiamo definire ricorsivamente il costo di $Sol(i, \ell)$:

$$C(i, \ell) = \begin{cases} 0 & \text{se } \ell < i \\ \max\{ C(i+1, \ell),\ P_i + C(i, \ell - i) \} & \text{altrimenti} \end{cases}$$

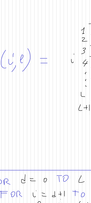
$C(2,6) = \begin{cases} C(3,6) \\ P_2 + C(3,3) \end{cases}$

$C(4,7) = \begin{cases} C(5,7) \\ P_4 + C(4,3) \end{cases}$

Otteniamo un n° quadratico $n \cdot L$ di sottoproblemi, ma ogni scelta richiede Tempo costante

Dobbiamo riempire la matrice $C[i, j]$

Un modo efficiente è il seguente:



Matrice riempita diagonale per diagonale

```
FOR d = 0 TO L
  FOR i = L+1 TO L+1
    ℓ = i - (L+1)
    C[i,ℓ] = 0
```
Inizializza a 0 le diagonali sotto quella principale

```
FOR d = 1 TO L DO
  FOR i = 1 TO L-d+1 DO
    ℓ = i + d - 1
    P1 = C[i+1, ℓ]
    P2 = P[i] + C[i, ℓ-i]
    IF P1 > P2 THEN
      C[i,ℓ] = P1
      S[i,ℓ] = 0
    ELSE
      C[i,ℓ] = P2
      S[i,ℓ] = 1
```
- d itera su diagonali
- i itera su righe
- ℓ indica la colonna

C riempita per diagonali

Il più piccolo Taglio è > L

Il più piccolo Taglio è = i

```
PRINT-SOL (L, S[,])
  ℓ = L
  i = 1
  WHILE ℓ > 0 DO
    IF S[i,ℓ] = 0 THEN
      i = i+1
    ELSE
      PRINT i
      ℓ = ℓ - i
```

Entrambe le decomposizioni portano a un algoritmo con tempo $O(L^2)$

In alternativa:

```
FOR i = L+1 TO 1 DO
  FOR ℓ = i-1 TO L DO
    C[i,ℓ] = 0
    S[i,ℓ] = 0
```
Inizializzazione

```
FOR i = L DOWNTO 1 DO
  FOR ℓ = i TO L DO
    P1 = C[i+1, ℓ]
    P2 = C[i, ℓ-i]
    IF P1 > P2 THEN
      C[i,ℓ] = P1
      S[i,ℓ] = 0
    ELSE
      C[i,ℓ] = P2
      S[i,ℓ] = 1
```

SCHEDULAZIONE DI INTERVALLI PESATI

Input Un insieme di attività (task) $T = \{t_1, \dots, t_n\}$

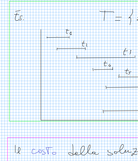
Ogni Task t_i ha tempo inizio $S(t_i)$ e Tempo fine $F(t_i)$

$(S(t_i), F(t_i))$ intervallo di t_i

Una funzione w $T \to \mathbb{N}$ $(w(i)$ valore del Task $t_i)$

Output Insieme di Task di valore massimo (somma dei valori) tale che nessun Task si sovrapponga agli altri

Es



Il costo della soluzione $S = \{t_{i_1}, \dots, t_{i_k}\}$ è

$C(S) = \sum_{j=1}^{k} w_{i_j} = w_{i_1} + \dots + w_{i_k}$

Soluzione $S = \{t_{i_1}, \dots, t_{i_k}\}$ per il problema

$S' = \{t_{i_1}, \dots, t_{i_{k-1}}\}$ il quale problema è sol ?

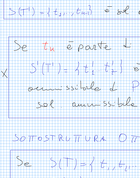
L'insieme dei Task a disposizione (intervalli) è un parametro del problema

Probl. $P(T)$ Con Sol $S(T) = \{t_{i_1}, \dots, t_{i_k}\}$

è ammissibile se per ogni coppia t_{i_r}, t_{i_s} in $S(T)$ con $r<s$ $\quad t_{i_r} \cap t_{i_s} = \emptyset$ [no sovrapposizione]

$t \cap t' = \emptyset$ sse $F(t) \le S(t')$ o $F(t') \le S(t)$

Es



$\{3, 2, 1\}$ è ammissibile?

$\{3, 1, 4, 5\}$ è ammissibile ?

Dato T e t_i definiamo

$T^i = \{t \in T \mid t \cap t_i = \emptyset\}$

(insieme di Task compatibili con t_i)

Allora:

- $S(T) = \{t_i, t_{i_2}, \dots, t_{i_k}\}$ è sol ammissibile per $P(T)$, $S(T^i) = \{t_{i_2}, \dots, t_{i_k}\}$ è sol ammissibile per $P(T^i)$.

- Se t_i è parte di sol di $P(T)$ e $S'(T^i) = \{t_{i_2}, t_{i_3}, \dots, t_{i_k}\}$ è sol ammissibile di $P(T^i)$, $\{t_i, t_{i_2}, t_{i_3}, \dots, t_{i_k}\}$ è sol ammissibile di $P(T)$???

SOTTOSTRUTTURA OTTIMA

Se $S(T) = \{t_i, t_{i_2}, \dots, t_{i_k}\}$ è sol. ottima per $P(T)$, $S(T^i) = \{t_{i_2}, \dots, t_{i_k}\}$ è sol ottima per $P(T^i)$???

PROBLEMA Quanti sono i possibili sottoproblemi di $P(T)$?

[I possibili sottoinsiemi di T]

⇓

n° esponenziale di sottoproblemi

2 CASI POSSIBILI:

1) Il problema non è risolvibile in Tempo polinomiale

2) Le formulazione non è adeguata (Più ossia troppo lenta)

Per il problema di Schedulazione di Intervalli siamo nel caso 2

Assumiamo che i Task (intervalli) siano ordinati rispetto al Tempo di Terminazione

$T = \{t_1, t_2, \dots, t_n\}$ t.c. $F(t_i) \le F(t_{i+1})$ $\forall i < n$

[possiamo ordinare T in tempo $\Theta(n \log n)$]

Definiamo una funzione

(A) $p(i) = \max\{j < i \mid F(j) < S(i)\}$

Se t_i è il primo Task nell'ordine, $p(i)$ è l'indice $< i$ dell'ultimo Task nell'ordine che Termina Prima che t_i inizi.

Qual è la funzione $p(\cdot)$ dell'esempio?

Possiamo ora riformulare la decomposizione

$P(n)$: Problema di schedulare gli n Task $t_1, \dots, t_n$ ordinati per tempo di terminazione

$S(n)$: Soluzione del problema $P(n)$

1) $S(n) = \{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}$ è ammissibile per $P(n)$ se

- $i_1 < i_2 < \dots < i_k < n$
- $F(t_{i_j}) \le S(t_{i_{j+1}}) \quad \forall 1 \le j < k$

Es 1 Se $S(n) = \{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}$ è ammissibile per $P(n)$, allora vale uno dei seguenti casi:

1) $i_k = n$ e $\{t_{i_1}, \dots, t_{i_{k-1}}\}$ è ammissibile per $P(p(n))$

2) $i_k < n$ e $\{t_{i_1}, \dots, t_{i_k}\}$ è ammissibile per $P(n-1)$

$n \notin S(n)$ → $n \notin S(n)$

Dim. Assumiamo 1) $i_k = n$ (cioè $n \in Sol(n)$) allora, per cui è ammissibile per $P(n)$

- $1 \le i_1 < i_2 < \dots < i_{k-1} < i_k = n$
- $F(t_{i_j}) \le S(t_{i_{j+1}}) \quad \forall 1 \le j < k$

In particolare $F(t_{i_{k-1}}) \le S(t_{i_k}) = S(n)$

⇓ (per definizione di $p(n)$)

$i_{k-1} \le p(n) < p(i_k)$

cioè allora $1 \le i_1 < i_2 < \dots < i_{k-1} \le p(n) < p(i_k)$

Quindi $\{t_{i_1}, t_{i_2}, \dots, t_{i_{k-1}}\}$ è ammissibile per $P(p(n)) = P(F(i_k))$

Assumo 2), allora $i_k < n$ e

$1 \le i_1 < i_2 < \dots < i_{k-1} < i_k < n$

$F(t_{i_j}) \le S(t_{i_{j+1}}) \quad \forall 1 \le j < k$

$1 \le i_1 < i_2 < \dots < i_{k-1} < i_k < n$

$F(t_{i_j}) \le S(t_{i_{j+1}}) \quad \forall 1 \le j < k$

⇓

$\{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}$ risulta è ammissibile per $P(n-1)$

Teo 2 Entrambe le seguenti sono vere:

1) se $i=n$ e $\langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(\ell(i))$, allora $\langle i_1, i_2, \dots, i_k, n \rangle$ è ammissibile per $P(S_i)$;

2) se $i<n$ e $\langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(i-1)$, allora $\langle i_1, i_2, \dots, i_k \rangle$ è ammissibile per $P(i)$.

Dim. In 2), se $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(\ell(i))$, allora

$$\begin{cases} i_1 < i_2 < \dots < i_k < \ell(i) \\ \ell(i_j) < s(i_j) \quad \forall\, 1 \le j \le k \end{cases}$$

Ma $\ell(i) < s(i)$ per def di $\ell(i)$

Quindi $\begin{cases} i_1 < i_2 < \dots < i_k < i_{j+1} < i \\ \ell(i_j) < s(i_j) \quad \forall\, 1 \le j \le k+1 \end{cases}$

Ma allora $\langle i_1, i_2, \dots, i_{j+1} \rangle$ (in quanto $\langle i_1, \dots, i_k \rangle$ è ammissibile per $P(\ell(i))$

Il caso 2) è ovvio (*perché?*)

SOTTOSTRUTTURA OTTIMA

Teo 3 Se $\langle i_1, i_2, \dots, i_{k-1}, i_k \rangle$ è sol ottima per $P(n)$, allora una delle seguenti vale:

1) $i_k = n$ e $\langle i_1, i_2, \dots, i_{k-1} \rangle$ è sol ottima per $P(i_{k-1})$

2) $i_k = n$ e $\langle i_1, i_2, \dots, i_k \rangle$ è sol ottima per $P(\ell(n))$

Dim. Nel caso 1), assumiamo per assurdo che $S = \langle i_1, i_2, \dots, i_k \rangle$ non sia ottima per $P(i_k)$. Ma allora $\exists\ S' = \langle i'_1, i'_2, \dots, i'_h \rangle$ tale che $c(S') < c(S)$ e $\langle i'_1, i'_2, \dots, i'_h, i_k \rangle$ è ammissibile per $P(n)$, perché S' è anche sol ammissibile per $P(i_k)$, otteniamo una contraddizione con la premessa del teorema.

Per il caso 2), assumiamo per assurdo che $S(\ell(n)) = \langle i_1, i_2, \dots, i_k \rangle$ non sia ottima per $P(\ell(n))$ $\Rightarrow \exists\ S' (\ell(n)) = \langle i'_1, i'_2, \dots, i'_h \rangle$ tale che $c(S') < c(S)$

Ma per Teo precedente, $S = \langle i_1, i_2, \dots, i_k, n \rangle$ è ammissibile per $P(n)$

$$\begin{aligned} C(S) &= v(i_1) + \dots + v(i_{k-1}) + v(n) = C(S(\ell(n))) + v(n) \\ &< C(S'(\ell(n))) + v(n) = C(S) \end{aligned}$$

che contraddice premessa del Teorema.

2) Costo di Soluzione ottima

$S(n) = \langle i_1, i_2, \dots, i_k \rangle$ ottima per $P(n)$

1) $i_k = n$ e $\langle i_1, i_2, \dots, i_{k-1} \rangle$ sol ottima per $P(\ell(i_k))$

$$C(S(n)) = \sum_{j=1}^{k} v(i_j) = v(i_k) + \sum_{j=1}^{k-1} v(i_j) = v(n) + C(S(\ell(i_k)))$$

Oppure

2) $i_k < n$ e $\langle i_1, \dots, i_k \rangle$ è sol ottima per $P(n-1)$

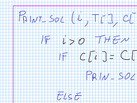
Quindi:

$$C(n) = \max\{v(n) + C(\ell(n)), C(n-1)\}$$

Funzione di costo ottimo:

$$C(i) = \begin{cases} 0 & \text{Se } i = 0 \\ \max\{v(i) + C(\ell(i)), C(i-1)\} & \text{altrimenti} \end{cases}$$

3) Calcolo Costo Soluzione Ottima



$C[i]$ dipende solo da valori $C[x]$ con $x < i$

```
Costo_Ottimo (n, v[], T[], ℓ[])
  C[0] = 0
  For i = 1 TO n DO
      C[i] = C[i-1]
      IF v[T[i]] + C[ℓ[i]] > C[i] THEN
          C[i] = v[T[i]] + C[ℓ[i]]
```

$\Theta(n)$

4) Costruzione Soluzione Ottima

Il seguente algoritmo ricorsivo fornisce la sequenza ottima di attività per $P(i)$.

```
PRINT_SOL (i, T[], C[], ℓ[])
  IF i > 0 THEN
      IF C[i] = C[i-1] THEN
          PRINT_SOL (i-1, T, C)
      ELSE
          PRINT_SOL (ℓ(i), T, C)
          Print T[i]
```

Tempo di soluzione è $\Theta(n)$, assumendo che:

1) T ordinato e

2) $\ell(i)$ calcolabile in tempo $\Theta(1)$

Per garantire ciò, possiamo:

1) Ordinare T costo $\Theta(n \log n)$

2) Precalcolare $\ell()$ [in un array] al costo $O(n \log n)$ [*perché??*]

Quindi il costo totale di Soluz. è $\Theta(n \log n)$

Per precalcolare la funzione $\ell()$ per ogni $i \le n$ possiamo usare la ricerca binaria in T.

```
BIN_SEARCH (i, T)
  p = 1
  r = i-1
  WHILE p ≤ r DO
      q = ⌊(p+r)/2⌋
      IF f[T[q]] < s[T[i]] THEN
          p = q+1
      ELSE
          r = q-1
  RETURN r
```

Per ogni task calcolare $\ell(i)$ costo $O(\log n)$

Riempiamo array $\ell[]$ con BIN_SEARCH(i, T) Per ogni Task i

```
COMPUTE_ℓ (T, n)
  For i = 1 TO n DO
      ℓ[i] = BIN_SEARCH (i, T)
```

per un costo di $O(n \log n)$.

Massima Sottosequenza Comune

Date due sequenze
$X = x_1 x_2 \dots x_n$
$Y = y_1 y_2 \dots y_m$
trovare la [Max Length Subsequence problem](LCS) di X e Y

Es:
$X = \underbrace{A}_{1}\,\underbrace{G}_{2}\,\underbrace{C}_{3}\,\underbrace{B}_{4}\,\underbrace{A}_{5}\,\underbrace{A}_{6}\,\underbrace{D}_{7}\,\underbrace{A}_{8}\,\underbrace{B}_{9}$
$Y = \underbrace{C}_{1}\,\underbrace{A}_{2}\,\underbrace{D}_{3}\,\underbrace{G}_{4}\,\underbrace{B}_{5}\,\underbrace{D}_{6}\,\underbrace{A}_{7}\,\underbrace{E}_{8}\,\underbrace{C}_{9}\,\underbrace{A}_{10}\,\underbrace{B}_{11}$
$Z_1 = \underbrace{A\,G\,B\,A\,A\,B}_{6}$

$X = \underbrace{A}_{1}\,\underbrace{G}_{2}\,\underbrace{C}_{3}\,\underbrace{B}_{4}\,\underbrace{A}_{5}\,\underbrace{A}_{6}\,\underbrace{D}_{7}\,\underbrace{A}_{8}\,\underbrace{B}_{9}$
$Y = \underbrace{C}_{1}\,\underbrace{A}_{2}\,\underbrace{D}_{3}\,\underbrace{G}_{4}\,\underbrace{B}_{5}\,\underbrace{D}_{6}\,\underbrace{A}_{7}\,\underbrace{E}_{8}\,\underbrace{C}_{9}\,\underbrace{A}_{10}\,\underbrace{B}_{11}$
$Z_2 = \underbrace{A\,G\,B\,A\,A\,B}_{6}$

$X = \underbrace{A}_{1}\,\underbrace{G}_{2}\,\underbrace{C}_{3}\,\underbrace{B}_{4}\,\underbrace{A}_{5}\,\underbrace{A}_{6}\,\underbrace{D}_{7}\,\underbrace{A}_{8}\,\underbrace{B}_{9}$
$Y = \underbrace{C}_{1}\,\underbrace{A}_{2}\,\underbrace{D}_{3}\,\underbrace{G}_{4}\,\underbrace{B}_{5}\,\underbrace{D}_{6}\,\underbrace{A}_{7}\,\underbrace{E}_{8}\,\underbrace{C}_{9}\,\underbrace{A}_{10}\,\underbrace{B}_{11}$
$Z_3 = \underbrace{C\,B\,D\,A\,B}_{5}$

$P(i,j)$ = LCS delle sequenze X_i e Y_j

$\begin{cases} X = x_1 x_2 \dots x_n \\ Y = y_1 y_2 \dots y_m \end{cases}$ $\quad Z = z_1 z_2 \dots z_k$

Sol ammissibile per $P(n,m)$

Condizione di Ammissibilità

Motivazione: cerco la soluzione $Z = z_1 z_2 \dots z_k$ per $P(n,m)$ cominciando a un matching (parzialmente)

$M_z = (i_z, j_z) \dots (i_{z'}, j_{z'})$ t.c.
a) $1 \le i_z < i_{z'} < \dots < i_{z''} \le n$
b) $1 \le j_z < j_{z'} < \dots < j_{z''} \le m$
c) $\forall$ indice ℓ $\quad z_\ell = x_{i_\ell} = y_{j_\ell}$

Es: $P(9,10)$:

$X = \underbrace{A}_{1}\,\underbrace{G}_{2}\,\underbrace{C}_{3}\,\underbrace{B}_{4}\,\underbrace{A}_{5}\,\underbrace{A}_{6}\,\underbrace{D}_{7}\,\underbrace{A}_{8}\,\underbrace{B}_{9}$
$Y = \underbrace{C}_{1}\,\underbrace{A}_{2}\,\underbrace{D}_{3}\,\underbrace{G}_{4}\,\underbrace{B}_{5}\,\underbrace{D}_{6}\,\underbrace{A}_{7}\,\underbrace{E}_{8}\,\underbrace{C}_{9}\,\underbrace{A}_{10}\,\underbrace{B}_{11}$
$Z = \underbrace{A\,G\,B\,A\,A\,B}_{6}$

$(1,2)\ (2,4)\ (3,5)\ (4,7)\ (5,8)\ (6,9)\ (7,10)$
$(1,1)\ (2,3)\ (3,4)\ (4,6)\ (5,7)\ (6,8)\ (7,10)$

E.C.
$\begin{cases} x_1 = x_2 = y_2 = y_4 = y_7 \\ x_3 = x_4 = y_5 = B = y_8 \\ x_5 = x_6 = y_9 = D = y_6 \\ \vdots \end{cases}$

1) CARATTERIZZARE Soluzione Ottima

$P(7,8)$:
$X = A\,B\,C\,B\,A\,B\,C$
$Y = B\,D\,A\,C\,B$
$Z_1 = B\,D\,B$ $\quad Z_1 \neq x_7$
$\Rightarrow P(\underline{6,7}) = P(6,6)$ $\quad \forall_{6,6}: (6,6) \in (C)$

$P(7,5)$:
$X = A\,B\,C\,B\,A\,B\,C$
$Y = B\,D\,A\,C\,B$
$Z_2 = B\,D\,C$ $\quad Z_2 \neq y_5$
$\Rightarrow P(\underline{6,5}) = P(\underline{7,4}) = P(6,4)$

$P(6,5)$:
$X = A\,B\,C\,B\,A\,B$
$Y = B\,D\,A\,C\,B$
$Z_3 = B\,C\,A\,B$ $\quad Z_3 \neq x_6 = y_5$
$\Rightarrow P(\underline{5,4}) = P(\underline{6,3}) = P(5,3)$

TEOREMA 1

Decomposizione di $Z = z_1 \dots z_k$ ammissibile per $P(i,j)$:

1) $z_k = x_i = y_j \implies z_1 \dots z_{k-1}$ è ammissibile per $P(i-1, j-1)$
2) $z_k \neq x_i \implies z_1 \dots z_k$ è ammissibile per $P(i-1, j)$
3) $z_k \neq y_j \implies z_1 \dots z_k$ è ammissibile per $P(i, j-1)$

Dim

1) Per assurdo esiste $M_{Z'} = (i_1, j_1) \dots (i_{k-1}, j_{k-1})$ con
a) $1 \le i_1 < i_2 < \dots < i_{k-1} \le i-1$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} \le j-1$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k-1$

Ma $i_{k-1} < i$ e $j_{k-1} < j$, quindi $i_{k-1} \le i-1$ e $j_{k-1} \le j-1$, per questi due e c) $\implies z_1 \dots z_{k-1}$ è ammissibile per $P(i-1, j-1)$ $\quad \forall\ \ell \le k-1$

Da questi due e c) con $\ell = k-1$ otteniamo che $z_k = x_{i-1} = y_{j-1}$ $\implies z_k = x_i = y_j$ $\implies$ z non è ammissibile per $P(i,j)$.

2) Per assurdo esiste $M_{Z'} = (i_1, j_1) \dots (i_{k-1}, j_{k-1})$ con
a) $1 \le i_1 < i_2 < \dots < i_{k-1} \le i-1$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} \le j-1$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k-1$

Inoltre se $z_k \neq x_i \implies i_{k-1} \neq i \implies i_{k-1} < i$
Quindi $M_{Z'} = (i_1, j_1) \dots (i_{k-1}, j_{k-1})$ è t.c.
a) $1 \le i_1 < i_2 < \dots < i_{k-1} < i \implies i_{k-1} \le i-1$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} \le j-1$ (da 1.)
c) è vera per assunzione.
$\implies z$ è ammissibile per $P(i-1, j)$.

3) analogo a 2)

TEOREMA 2

1) Se $z_1 \dots z_k$ è sol ammissibile per $P(i-1, j-1)$ e $x_i = y_j$, allora $z_1 \dots z_k x_i$ (cioè $z_{k+1} = x_i = y_j$) è ammissibile per $P(i,j)$

2) Se $z_1 \dots z_k$ è ammissibile per $P(i-1, j)$, allora $z_1 \dots z_k$ è ammissibile per $P(i, j)$

3) Se $z_1 \dots z_k$ è ammissibile per $P(i, j-1)$, allora $z_1 \dots z_k$ è ammissibile per $P(i, j)$

Dim

1) Se $z = z_1 \dots z_k$ è ammissibile per $P(i-1, j-1)$ allora $M_Z = (i_1, j_1) \dots (i_{k-1}, j_{k-1})$ è t.c.
a) $1 \le i_1 < i_2 < \dots < i_{k-1} \le i-2 < i-1$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} \le j-2 < j-1$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k-1$

Inoltre $x_i = y_j$, quindi posto $z_{k+1} = x_i = y_j$, $i_k = i$ e $j_k = j$ otteniamo che $Z' = z_1 \dots z_k z_{k+1}$ e $M_{Z'} = M_Z \cup \{(i_k, j_k)\} = \{(i_1, j_1) \dots (i_{k-1}, j_{k-1}), (i_k, j_k)\}$ è t.c.
a) $1 \le i_1 < i_2 < \dots < i_{k-1} < i_k = i$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} < j_k = j$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k$
$\implies z'$ è ammissibile per $P(i, j)$

2) e 3) sono molto semplici e lasciati come esercizio

SOTTOSTRUTTURA OTTIMA

TEOREMA 3

$Z = z_1 \dots z_k$ sol ottima per $P(n,m)$, allora:

1) $x_n = y_m \implies$ $z_1 \dots z_{k-1}$ è sol ottima per $P(n-1, m-1)$ (1.a)

2) $x_n \neq y_m \implies$ 2.1) $z_k \neq x_n \implies z_1 \dots z_k$ è ottima per $P(n-1, m)$ (2.b)

2.2) $z_k \neq y_m \implies z_1 \dots z_k$ è ottima per $P(n, m-1)$

Dim

1.a) Per assurdo assumiamo che
(*) $z_k \neq x_n$ (e quindi $z_k \neq y_m$, poiché $x_n = y_m$).
$M_Z = (i_1, j_1) \dots (i_{k-1}, j_{k-1})$ t.c.
a) $1 \le i_1 < i_2 < \dots < i_{k-1} \le n-1 < n$
b) $1 \le j_1 < j_2 < \dots < j_{k-1} \le m-1 < m$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k$

Ma allora, per (*) e c), abbiamo che
$i_{k-1} \neq n \implies i_{k-1} < n$
$j_{k-1} \neq m \implies j_{k-1} < m$
Ma $x_n = y_m$, quindi posto $\begin{cases} z_{k+1} = x_n = y_m \\ i_{k+1} = n \\ j_{k+1} = m \end{cases}$

Possiamo costruire la sequenza $Z' = z_1 \dots z_k z_{k+1}$ corrispondente al matching
$M_{Z'} = (i_1, j_1) \dots (i_{k+1}, j_{k+1})$
che soddisfa i vincoli:
a) $1 \le i_1 < i_2 < \dots < i_{k+1} = n$
b) $1 \le j_1 < j_2 < \dots < j_{k+1} = m$
c) $z_\ell = x_{i_\ell} = y_{j_\ell}$ $\quad \forall\ \ell \le k+1$

Quindi Z' è ammissibile per $P(n, m)$ ed è più lunga di Z $\implies$ Z non ottima $\rightarrow$ Contraddizione!!

1.b) Assumiamo $z_1 \dots z_{k-1}$ non ottimo

Per $P(i-1, j-1)$, allora $\exists\, z^*$

$z^* = z_1^* \dots z_r^*$ ammissibile per $P(i-1, j-1)$

con $r > k-1$ e

$M_{z^*} = (i_1^*, j_1^*) \dots (i_r^*, j_r^*)$ t.c.

a) $1 \le i_1^* < \dots < i_r^* \le i-1$

b) $1 \le j_1^* < \dots < j_r^* \le j-1$

c) $z_\ell^* = x_{i_\ell} = y_{j_\ell}$ $\forall\, \ell = 1 \dots r$

Ma allora, posto:

$z_{r+1}^* = x_i = y_j$, $z_1^* \dots z_r^* z_{r+1}^*$, $i_{r+1}^* = i$, $j_{r+1}^* = j$

il matching $(i_1^*, j_1^*) \dots (i_r^*, j_r^*)(i_{r+1}^*, j_{r+1}^*)$

è ammissibile per $P(i,j)$

e ha lunghezza $r+1 > k$

$\Rightarrow$ z non è ottimo $\Rightarrow$ Contraddizione!!

2) e 3) lasciata come esercizio

2) **FUNZIONE DI COSTO OTTIMO**

$$C(i,j) = \begin{cases} 0 & \text{se } i = 0 \circ j = 0 \\ 1 + C(i-1, j-1) & \text{se } x_i = y_j \\ \max\{C(i-1, j), C(i, j-1)\} & \text{altrimenti} \end{cases}$$

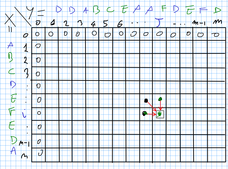
$P(n,m)$ richiede $n \cdot m$ sottoproblemi distinti.

Utilizzando matrice di dimensione $n \cdot m$ possiamo calcolare $C(i,j)$ in tempo costante e ottenere un algoritmo di costo complessivo $\Theta(n \cdot m)$

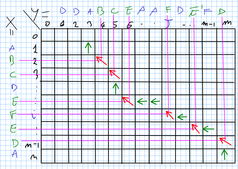
3) **CALCOLO COSTO OTTIMO**

X = A B C D E F E D A

Y = D D A B C E A A F D E F D



C[n,m]



S[n,m]

```
COSTO-OTTIMO (X[ ], Y[ ], n, m)
FOR j = 0 TO m DO
    C[0,j] = 0
FOR i = 1 TO n DO
    C[i,0] = 0
FOR i = 1 TO n DO
    FOR j = 1 TO m DO
        IF x[i] = y[j] THEN
            C[i,j] = 1 + C[i-1, j-1]
            S[i,j] = "↖"
        ELSE
            IF C[i-1,j] ≥ C[i,j-1] THEN
                C[i,j] = C[i-1,j]
                S[i,j] = "←"
            ELSE
                C[i,j] = C[i,j-1]
                S[i,j] = "↑"
```

4) **RICOSTRUZIONE SOLUZIONE OTTIMA**

```
PRINT_SOL (i, j, S[ ], X[ ])
    IF i > 0 ∧ j > 0 THEN
        IF S[i,j] = "↖" THEN
            PRINT_SOL (i-1, j-1, S, X)
            PRINT X[i]
        ELSE IF S[i,j] = "↑" THEN
            PRINT_SOL (i-1, j, S, X)
        ELSE
            PRINT_SOL (i, j-1, S, X)
```

ALBERO OTTIMO DI RICERCA

Date n chiavi $k_1, k_2, \dots k_n$ ordinate in modo non decrescente $k_1 \le k_2 \le \dots \le k_n$

e una distribuzione di probabilità sulle n chiavi:

p_i è la probabilità che la chiave k_i venga cercata.

Dato un albero binario di ricerca T caratterizzato da n chiavi: e detta $d_T(i)$ la profondità della chiave k_i in T,

$$C(T) = \sum_{i=1}^{n} \left(d_T(i)+1\right) * p_i$$ [Tempo medio di ricerca]

definiamo il costo medio di ricerca in T (assumendo che ogni ricerca abbia esito positivo, cioè che la chiave cercata sia presente nell'albero)

Problema: Costruire l'albero binario di ricerca che minimizza il tempo medio di ricerca

Esempio:

i	1	2	3	4	5
p_i	0,1	0,2	0,2	0,15	0,3

T_1 [albero bilanciato]

$$C(T_1) = (0+1)\cdot 0{,}2 + (1+1)\cdot 0{,}2 + (2+1)\cdot 0{,}15 + (2+1)\cdot 0{,}3 + (3+1)\cdot 0{,}15 = 2{,}1$$

T_2 [albero ottimo]

$$C(T_2) = (0+1)\cdot 0{,}1 + (1+1)\cdot 0{,}2 + (1+1)\cdot 0{,}2 + (2+1)\cdot 0{,}15 + (2+1)\cdot 0{,}3 = 2{,}05$$

i	1	2	3	4	5
p_i	0,1	0,2	0,2	0,15	0,3

T_3

$$C(T_3) = (0+1)\cdot 0{,}3 + (1+1)\cdot 0{,}1 + (2+1)\cdot 0{,}2 + (2+1)\cdot 0{,}15 + (3+1)\cdot 0{,}2 = 2{,}15$$

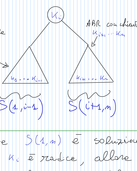
[chiavi in maggior probabilità più in alto]

$$C(T) = \sum_{i=1}^{n} \left(d_T(i)+1\right) * p_i$$

1) CARATTERIZZAZIONE SOLUZIONE OTTIMA

Ogni soluzione ammissibile deve essere un albero binario di ricerca che contiene tutte le chiavi $k_1, \dots k_n$

Tale albero avrà una radice k_r e forma:

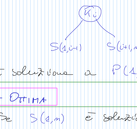


Teorema 1

Se $S(1,n)$ è soluzione al problema $P(1,n)$ e k_r è radice, allora $S(1, r-1)$ e $S(r+1, n)$ sono soluzioni ai problemi $P(1, r-1)$ e $P(r+1, n)$, rispettivamente

Teorema 2:

Se $S(1, r-1)$ e $S(r+1, n)$ sono sol per $P(1, r-1)$ e $P(r+1, n)$, rispettivamente, allora



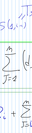
è soluzione a $P(1,n)$

SOTTOSTRUTTURA OTTIMA

Teorema 3

Se $S(1,n)$ è soluzione ottima per $P(1,n)$ e k_r è la radice, allora $S(1, r-1)$ e $S(r+1, n)$ [i sottoalberi sx e dx di k_r] sono soluzioni ottime per $P(1, r-1)$ e $P(r+1, n)$, rispettivamente

Dim Detto $T = S(1,n)$



$$C(T) = \sum_{j=1}^{n} \left[\left(d_T(j)+1\right) p_j\right] = \sum_{j=1}^{n} p_j + \sum_{j=1}^{n} \left(d_T(j)\cdot p_j\right) =$$

$$= \sum_{j=1}^{n} p_j + \sum_{j=1}^{r-1} \left(d_T(j)\, p_j\right) + d_T(r)\, p_r + \sum_{j=r+1}^{n} \left(d_T(j)\, p_j\right)$$

$$= \sum_{j=1}^{n} p_j + \sum_{j=1}^{i-1} \left[\left(d_{T_{sx}}(j)+1\right) p_j\right] + \sum_{j=i+1}^{n} \left[\left(d_{T_{dx}}(j)+1\right) p_j\right]$$

$d_T(j) = d_{T_{sx}}(j) + 1 \quad \forall\, 1 \le j \le i-1$

$d_T(j) = d_{T_{dx}}(j) + 1 \quad \forall\, i+1 \le j \le n$

$$= \sum_{j=1}^{n} p_j + C\left(T_{sx}\right) + C\left(T_{dx}\right) = \sum_{j=1}^{n} p_j + C\left(S(1, r-1)\right) + C\left(S(r+1, n)\right)$$

$$C\left(S(1,n)\right) = \sum_{j=1}^{n} p_j + C\left(S(1, r-1)\right) + C\left(S(r+1, n)\right)$$

Il resto della dimostrazione segue il solito ragionamento per assurdo ed è lasciato come esercizio.

2) FUNZIONE DI COSTO OTTIMO

Per ogni sottoproblema $P(i,j)$ la scelta locale è la chiave da mettere in radice, usiamo $k_i, k_{i+1}, \dots, k_j$

Quindi la funzione di costo ottimo è:

$$C(i,j) = \begin{cases} 0 & \text{se } i > j \\ \min\limits_{i \le k \le j} \left\{ \sum\limits_{p=i}^{j} p_p + C(i, k-1) + C(k+1, j) \right\} & \text{altrimenti} \end{cases}$$

chiave in radice

$W(i,j)$ indipendente da k

Possiamo, quindi, $W(i,j) = \sum_{p=i}^{j} p_p$ possiamo scrivere:

$$C(i,j) = \begin{cases} 0 & \text{se } i > j \\ W(i,j) + \min\limits_{i \le k \le j} \left\{ C(i, k-1) + C(k+1, j) \right\} & \text{altrimenti} \end{cases}$$

3) CALCOLO DEL COSTO OTTIMO

$C(i,j) =$

	0	1	2	3	4	5	…	n
1	0							
2		0						
3			0					
4				0				
5					0			
⋮						⋱		
n							0	
n+1								0

```
COSTO_OTTIMO (n, P[])
  FOR i = 1 TO n+1 DO
      C[i, i-1] = 0
  FOR i = n TO 1 DO
      W = 0
      FOR j = i TO n DO
          W = W + Pj                    // W(i,j) = Σ_{p=i}^{j} Pp  */
          C = 0
          FOR k = i TO j DO
              q = W + C[i, k-1] + C[k+1, j]
              IF C < q THEN
                  C = q
                  S = k
      C[i, j] = C
      S[i, j] = S
```

4) COSTRUZIONE ALBERO OTTIMO

Lasciato come esercizio

PROBLEMA DELLO ZAINO

Siano dati: n oggetti $O_1 O_2 \dots O_n$ ciascuno con un peso p_i e un valore v_i. Sia dato un contenitore (lo zaino) che può sostenere al massimo un peso W (capienza).
Calcolare l'insieme di oggetti di valore massimo che lo zaino è in grado di contenere.
(valore dello zaino è inteso come la somma dei valori degli oggetti in esso contenuti.)

Percorsi Minimi in Grafi Pesati: Algoritmo di Bellman-Ford

Dato un grafo $G=(V,E)$ - di cui una funzione di peso $w: E \to \mathbb{R}$ è data (con $u, v \in V$, $(u,v) \in E$, $w(u,v)$ è il peso associato all'arco (u,v) - e due vertici $s, t \in V$, individuare un percorso di peso minimo (a cui possono essere pesi positivi, nulli o negativi) da s a t.

Pesi: un percorso $\pi = \langle v_0, v_1, \ldots, v_k \rangle$ (di lunghezza k) ha peso $w(\pi) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$ - la somma dei pesi degli archi in π.

Nota: Se il grafo contiene **cicli** a peso negativo, il problema non ammette soluzione.

Possiamo definire $P(s,t)$ come il problema di calcolare il percorso minimo da s a t in G.

Una sottostruttura $S = \langle i, j, \ldots, m, n \rangle$ è ammissibile per $P(s,t)$ se:
1) $s = i$ e $n = t$
2) $(v_j, v_k) \in E \quad \forall j, k$

Ogni soluzione ammissibile per $P(s,t)$ è un percorso da s a t.

Una soluzione ammissibile per $P(s,t)$ è un percorso minimo se la sua lunghezza è minima tra quelle di tutte le soluzioni ammissibili per $P(s,t)$.

1) $S(s,x) = \langle s, \ldots, x \rangle$ Sia ammissibile per $P(s,x)$

$S(s,y) = \langle s, \ldots, x, y \rangle$ ammissibile per $P(s,y)$ se e solo se $S(s,x)$ è ammissibile per $P(s,x)$ e $(x,y) \in E$

Sono soluzioni ammissibili per $P(s,x)$ e $P(s,y)$, rispettivamente.

La dimostrazione è ovvia, poiché ogni percorso è costituito da percorsi (eventualmente vuoti) di un percorso e un arco.

2) Se $S(s,x) = \langle s, \ldots, u, x \rangle$ e $S(s,y) = \langle s, \ldots, v, y \rangle$ sono entrambi soluzioni ammissibili per $P(s,x)$ e $P(s,y)$, rispettivamente, allora:

$S' = S(s,x) \cdot S(x,y) = \langle s, \ldots, u, x, y \rangle$

è ammissibile per $P(s,y)$.

Anche questo è ovvio, poiché ho concatenato due percorsi, tali che il primo termina nel vertice da cui inizia il secondo, e un percorso può formare un ciclo.

Possiamo considerare anche le seguenti varianti di decomposizione in due problemi sempre più "lievi":

$S(s,t) = \langle s, \ldots, u, x, \ldots, t \rangle \Rightarrow \begin{cases} S(s,x) = \langle s, \ldots, u, x \rangle \\ S(x,t) = \langle x, \ldots, t \rangle \end{cases}$

$S(s,t) = \langle s, \ldots, u, x, y, \ldots, t \rangle \Rightarrow \begin{cases} S(s,x) = \langle s, \ldots, u, x \rangle \\ S(x,y) = \langle x, y \rangle \\ S(y,t) = \langle y, \ldots, t \rangle \end{cases}$

La proprietà di decomposizione delle soluzioni ottime descrive anche l'ottimalità ricorsiva.

Problema: Un decomposizione data spesso non è semplice da definire.

Es

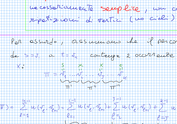
$P(s,x) \Rightarrow (s,x) \cdot P(x,t) \Rightarrow (s,x) \cdot (x,t) \cdot P(t,s)$

La ricorsione non è ben definita e non garantisce terminazione poiché può richiedere di risolvere $P(s,t)$ a partire da se stesso, ovvero il problema $P(s,t)$ richiede come primo sottoproblema il problema $P(s,t)$ stesso: $P(s,t)$ richiede di se stesso.

Osservazione 1: Il problema sempre quando il grafo G in input è aciclico. Perché?

Osservazione 2: Un percorso semplice può avere al massimo $|V|-1$ archi.

Dim: Per assurdo, assumiamo che il percorso minimo π da s a t contenga $\geq$ occorrenze del vertice v:



$w(\pi) = \sum_{i=1}^{k} w(v_{i-1}, v_i) = \sum_{i=1}^{j} w(v_{i-1}, v_i) + \sum_{i=j+1}^{k} w(v_{i-1}, v_i)$

Abbiamo 3 casi:

a) Il peso dal sottopercorso π^{+} è negativo

$w(\pi^{+}) = \sum_{i=j+1}^{k} w(v_{i-1}, v_i) < 0$

In questo caso un arco percorso minimo da s a t [perché?]

b) Il peso dal sottopercorso π^{+} è non negativo

$w(\pi^{+}) = \sum_{i=j+1}^{k} w(v_{i-1}, v_i) \geq 0$

In questo caso il percorso $\pi' = \pi^{-}$, ottenuto eliminando il sottopercorso π^{+} da π, ha peso $w \leq w$. Quindi se π è ottimo, lo è anche π^{-}.

Osservazione 3: è possibile di concludere che un percorso minimo tra 2 vertici di un grafo $G=(V,E)$ ha al massimo lunghezza $|V|-1$ archi.

1) Caratterizzazione struttura soluzioni ottime

Possiamo allora definire il problema in questo modo:

$P(s,t,|V|-1)$ Problema di trovare il percorso minimo da s a t composto al più $|V|-1$ archi.

La decomposizione ricorsiva generica (quando $|V|=n$)

Sia $S(s,t,m-1) = \langle i, \ldots, j, k \rangle$ soluzione a $P(s,t,m-1)$:

caso 1) Se $k = s$, allora $S(s,t,m) = \langle i, \ldots, j, k, t \rangle$ con $k = s$

caso 2) Se $k \neq s$, allora $S(s,t,m) = S(s,t,m-1)$

Dobbiamo:
1) Per ottenere n un $\langle i, \ldots, j \rangle$ ottimo per $P(s,x,m-1)$, allora $S = \langle i, \ldots, j, x \rangle$ ammissibile per $P(s,t,m-1)$ [perché contenente al massimo $m-1$ archi.]
Vale che $w(S') < w(S(s,x,m))$
Ma $S' = S(s,x)$ è ammissibile per $P(s,x,m-1)$ e $w(S') = w(S(s,x)) + w(S(x,s,m-1)) < w(S(s,x,m))$ $\Rightarrow$ $S(s,x,m-1)$ non è ottimo $\Rightarrow$ assurdo
2) Dimostrato in maniera analoga.

2) Funzione di costo Ottimo

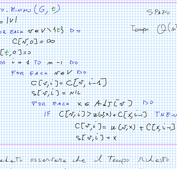
$$C(s,i) = \begin{cases} 0 & \text{se } s = i \\ \infty & \text{se } s \neq i \text{ e } i \text{ non ha archi entranti} \\ \min\{C(s,v) + w(v,i) : (v,i) \in E\} & \text{altrimenti.} \end{cases}$$

Il numero k di problemi distinti è $|V| \cdot |V| = n^2$

Ogni nuova scelta ottima di v per sottoproblemi (i,s,v) di varia n (assumiamo n di costi) $\Rightarrow$ richiede di più $|V|$ operazioni $\Rightarrow$ costo complessivo di $O(n^3)$

3) Calcolo del Cammino Ottimo

Come negli esempi precedenti: usiamo una tabella $C[i,j]$ di dimensione $|V| \times |V|$ per il costo. Il primo parametro i varia nell'intervallo $[0, |V|-1]$, il secondo parametro j varia nell'intervallo $[0, |V|-1]$. Il costo è dato dalla $C[i,j]$ dovrà solo dal valore di i e j, dalla tabella calcolata precedente e dalla colonna di candidati $C[x,i-1]$ con $x \in Adj[x]$.



$n = |V|$

Costo-Ruimo(G, s) Spazio $\Theta(n^2)$

$n = |V|$ Tempo $\Theta(n^2) \cdot O(n) = O(n^3)$

For each $v \neq V \setminus \{s\}$ do
$C[v,0] = \infty$
$C[s,0] = 0$
For $i = 1$ to $n-1$ do
For each $v \neq V$ do
$C[v,i] = C[v,i-1]$
$S[v,i] = nil$
For each $x \in Adj[v]$ do
If $C[v,i] > w(x,v) + C[x,i-1]$ Then
$C[v,i] = w(x,v) + C[x,i-1]$
$S[v,i] = x$

E' immediato osservare che il tempo richiesto è $O(|V|^3)$. Si può dimostrare che, in effetti, il tempo richiesto è $\Theta(|V| \cdot |E|)$. [Perché?]

Richiede spazio $\Theta(|V|^2)$, uno per ogni coppia di vertici che vengono considerati. Un ulteriore spazio $\Theta(|V|)$.

4) Cammino Percorso Minimo

- Se, al termine, troviamo che $C[v, |V|-1] = \infty$, allora non è un s raggiungibile da s
- Se $C[v, |V|-1] \neq \infty$, allora esiste un percorso da s a v. Il percorso minimo può essere estratto ricostruendolo dal ragionato all'indietro.

Percorso-Minimo(G, t, v, i, S[])
If $t \neq v$ Then
$x = S[v,i]$
If $x \neq nil$ Then
Print x
Percorso-Minimo(G, t, x, i-1, S[])
Else
Print "-1"
Else
Percorso-Minimo(G, t, v, i-1, S[])
Print v

Costo: $O(|V|)$

ALGORITMI GREEDY

PROGRAMMAZIONE DINAMICA: valuta tutte le possibili scelte locali, calcolando per ciascuna il miglior costo complessivo ottenibile. Il costo complessivo associato a una scelta è calcolato comparando il contributo locale di quella scelta con il costo ottimo del sottoproblema risultante dalla scelta stessa.
Tra queste scelte seleziona, poi, quella che fornisce il costo complessivo migliore.

Un **ALGORITMO GREEDY** si basa sull'idea di risolvere il problema costruendo la soluzione in modo incrementale.
Per ogni sottoproblema si considera sempre una sola scelta localmente ottima.

La valutazione della bontà della scelta è fatta unicamente considerando il sottoproblema corrente e le possibili scelte, quindi basandosi solo su informazioni locale.

Limitarsi a valutazioni locali della scelta tipicamente porta ad algoritmi più efficienti.

Non sempre la soluzione che si ottiene è ottima.

I problemi che ammettono soluzioni ottime tramite tecnica greedy soddisfano due condizioni:

1) SOTTOSTRUTTURA OTTIMA
2) PROPRIETÀ SCELTA GREEDY

La condizione 1) è la stessa richiesta per l'applicazione della PROGRAMMAZIONE DINAMICA.

La condizione 2), invece, richiede che la scelta greedy che viene effettuata per ogni sottoproblema, faccia necessariamente parte di una soluzione ottima a quel sottoproblema.

SCHEDULAZIONE INTERVALLI NON PESATI

Input: Un insieme di attività (task) $T = \{t_1, \dots, t_n\}$
Ogni Task i ha tempo inizio $S(i)$ e tempo fine $F(i)$ ($S(i) \le F(i)$) intervallo di i

Output: Insieme di Task di dimensione massima tale che nessun Task si sovrapponga agli altri.

Nota: Qui la condizione di ottimalità è determinata solo dal numero di intervalli (Task) schedulati, non dal loro valore complessivo.
Questo rende il problema più semplice da risolvere.

Come per la versione pesata del problema, assumiamo che gli intervalli siano già ordinati per tempo di terminazione $F(i)$ crescente:

$T = t_1, t_2 \dots t_n$ con $F(t_i) \le F(t_{i+1})$ $\forall$ i da 1 a $n-1$

Useremo la seguente notazione di problema e decomposizione

$P(i)$: Problema di schedulare gli intervalli (Task) da t_1 a t_i che sono compatibili con i $\forall i \ge 0$

Assumiamo un task fittizio 0 a $S(0) = F(0) = 0$, il problema principale è, quindi, $P(n)$.

DECOMPOSIZIONE:

- $S(n) = J_a\ J_b \dots J_m$ ammissibile per $P(n)$
  - $a < b < \dots < m < n$
  - $\forall$ i da a a m $\quad J_i \cap J_{i+1} = \emptyset$ [J_i e J_{i+1} sono compatibili]
  - $\forall\ i$ da a a m $\quad J_i \cap J_n = \emptyset$ [J_i e J_n sono compatibili]

Nota poiché i Task sono ordinati, se J_k è compatibile con J_{k+1} $[J_k \cap J_{k+1} = \emptyset]$ per ogni $1 \le j < k$, allora J_k è compatibile con tutti i J_m tali che $m \ge k$.

$\left[\begin{array}{l} J_k \cap J_{k+1} = \emptyset \Rightarrow F(J_k) < S(J_{k+1}) \\ J_{k+1} \cap J_{k+2} = \emptyset \Rightarrow F(J_{k+1}) < S(J_{k+2}) \\ \text{con } S(J_{k+1}) \le F(J_{k+1}) \Rightarrow F(J_k) < S(J_{k+2}) \\ \text{(per transitività)} \\ \text{e quindi } J_k \cap J_{k+2} = \emptyset \end{array} \right]$

Iterando il ragionamento si ottiene per tasi

In generale

$S(i) = J_a \dots J_m$ ammissibile per $P(i)$
se
- $a < b < \dots < m < n$
- $\forall$ i da a a m $\quad J_i \cap J_n = \emptyset$

1.- Se $J_a\ J_b \dots J_m$ è ammissibile per $P(i)$
allora $J_b \dots J_m$ è ammissibile per $P(J_a)$

2.- Se $J_a\ J_b \dots J_m$ è ammissibile per $P(i)$
e $J_a | J_b' \dots J_k'$ è ammissibile per $P(J_a)$
allora $J_b' \dots J_k'$ è ammissibile per $P(i)$

3.- Se $J_a\ J_b \dots J_m$ è ottima per $P(i)$
allora $J_b\ J_c \dots J_m$ è ottima per $P(J_a)$

Dimostrare le 3 proprietà di decomposizione sopra è lasciato come esercizio.

La decomposizione appena definita e il fatto che il costo di una soluzione è dato dal n° di intervalli (Task) che esse contiene, suggeriscono una funzione di costo ottimo definita come

$$C(i) = \begin{cases} 0 & \text{se } i = 0 \\ 1 + \max_{\substack{j < i \\ F(j) \le S(i)}} \{ C(j) \} & \text{altrim.} \end{cases}$$

Per ogni problema $P(i)$, vengono Testate Tutte le scelte $j < i$ che sono compatibili con i e, per ciascuna di esse, viene risolto il sottoproblema $P(j)$ corrispondente.

È possibile evitare il calcolare l'effetto di ogni possibile scelta (cioè risolvere tutti i sottoproblemi $P(j)$) e ottenere la scelta ottima per $P(i)$ confrontando tra di loro solo la possibile scelta?

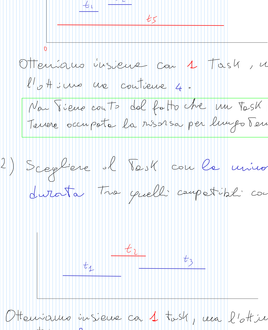
UNA SOLUZIONE GREEDY

Un Algoritmo Greedy utilizza una politica di scelta locale

Per ogni sottoproblema, effettua una sola scelta, quella che sembra la migliore, senza calcolare i costi ottimi complessivi subito di ciascuna scelta.

Possibili esempi:

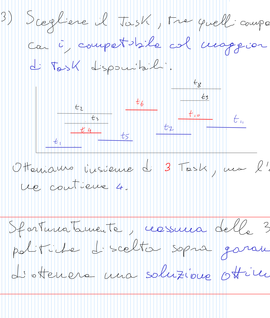
1) Scegliere sempre il Task che inizia primo tra quelli compatibili con i.



Otteniamo insieme con 1 Task, ma l'ottimo ne contiene 4.

Non tiene conto del fatto che un Task può tenere occupata la risorsa per lungo tempo

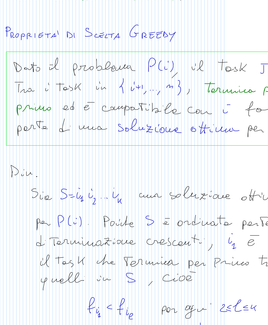
2) Scegliere il Task con la minor durata tra quelli compatibili con i.



Otteniamo insieme con 1 task, ma l'ottimo ne contiene 2.

Non tiene conto che un Task potrebbe essere incompatibile con molti Task

3) Scegliere il Task, tra quelli compatibili con i, compatibile col maggior n° di Task disponibili.



Otteniamo insieme di 3 Task, ma l'ottimo ne contiene 4.

Sfortunatamente, nessuna delle 3 politiche di scelta sopra garantisce di ottenere una soluzione ottima.

POLITICA GREEDY OTTIMA PER P(i)

Scegliere il Task, tra quelli compatibili con i, che Termina per primo.

Questa politica garantisce di ottenere sempre una soluzione ottima.

PROPRIETÀ DI SCELTA GREEDY

Dato il problema P(i), il Task J da tra i Task in $\{i_1, i_2, \dots, i_n\}$, Termina per primo ed è compatibile con i fa parte di una soluzione ottima per P(i)

Dim.

Sia $S = i_{j_1} i_{j_2} \dots i_{j_k}$ una soluzione ottima per P(i). Poiché S è ordinata per tempi di terminazione crescenti, i_{j_1} è il Task che termina per primo tra quelli in S, cioè

$$f_{j_1} \leq f_{j_2} \quad \text{per ogni } z \in \{1 \dots k\}$$

Sia J il Task in $\{i_1, i_2, \dots, i_n\}$ che Termina per primo ed è compatibile col Task i.

Ovviamente $J \leq i_{j_1}$ [Perché?]

Abbiamo 2 casi:

1) $J = i_{j_1}$, allora la tesi è verificata banalmente

2) $J \neq i_{j_1}$. Ma $J \leq i_{j_1} \Rightarrow J < i_{j_1}$. Poiché i Task sono ordinati per tempi di terminazione crescenti, vale $f[J] \leq f[i_{j_1}]$

Inoltre J è compatibile con i, quindi $f[J] < S[J]$

Sia $S^* = J\, i_{j_2} \dots i_{j_k}$.

S^* è ammissibile per P(i).

Infatti:

- $i < J < i_{j_2} < i_{j_3} < \dots < i_{j_k} \leq n$
- $J \cap i_{j_2} = \emptyset$ e $i_{j_2} \cap i_{j_m} = \emptyset \quad \forall z \in \{2 \dots k\}$
- $i \cap J = \emptyset$ [per scelta di J]

Ma il n° di Task in S^* è lo stesso che in S $\Rightarrow S^*$ è anch'essa ottima $\Rightarrow$ Tesi

□

ALGORITMO GREEDY

Il seguente algoritmo ottiene la soluzione ottima, facendo sempre la scelta greedy e sfruttando il fatto che i Task sono ordinati per Tempo di Terminazione (f[i]) crescente.

```
SCHEDULER (T[1..n])
    SOL = ∅
    f = 0
    FOR i = 1 TO n DO
        IF S[i] > f THEN
            SOL = SOL ∪ {i}
            f = f[i]
    RETURN SOL
```

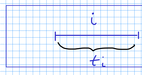
Tempo di esecuzione $\Theta(n)$, se l'operazione di unione (SOL ∪ {i}) è implementata a Tempo costante (e.g. inserimento in Testa alla lista SOL).

[$\Theta(n \log n)$ se includiamo ordinamento di T]

Scheduling con Minimo Tempo Medio di Completamento

Input: Un insieme di attività (task) $T = \{1, \dots, n\}$

Ogni Task i richiede tempo t_i per essere elaborato.
(t_i è la durata del Task i)



Output: Schedulazione di tutti gli n task che minimizza il tempo medio di completamento degli n task.

$T = \{1, \dots, n\}$

Sottostruttura Ottima

Consideriamo il problema $P(T, 0)$ di schedulare i Task in T a partire dal tempo 0 (zero).

$S = (s_1, i_1)\ (s_2, i_2) \cdots (s_n, i_n)$ è ammissibile per $P(T, 0)$

(s_ℓ: Tempo inizio Task i_ℓ ..... Tempo inizio Task i_n)

se

1) $0 \le s_\ell$
2) $s_\ell + t_{i_\ell} \le s_{\ell+1}$ ✓ $1 \le \ell \le n-1$
3) $\{i_1, i_2, \dots, i_n\} = T$ (S contiene tutti e soli i Task in T)

Costo di una soluzione $S = (s_1, i_1)\ (s_2, i_2) \cdots (s_n, i_n)$

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(\ell) \qquad \text{dove} \quad f(\ell) = s_\ell + t_{i_\ell}$$

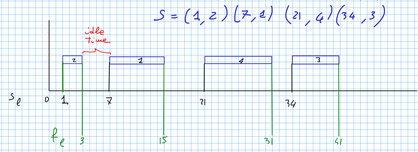
$\frac{1}{n}$: Tempo medio di terminazione

$f(\ell) = s_\ell + t_{i_\ell}$: Tempo di completamento dell'ℓ-esimo Task in S (Task i_ℓ)

Osservazione: Possiamo ignorare soluzioni con intervalli di inattività (idle Time).

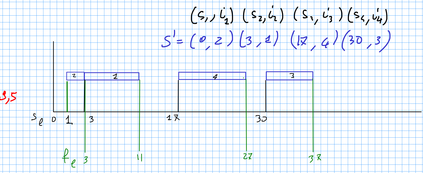
Task	durata
1	$(t_1 = 8)$
2	$(t_2 = 4)$
3	$(t_3 = 2)$
4	$(t_4 = 10)$

$(s_1, i_1)\ (s_2, i_2)\ (s_3, i_3)\ (s_4, i_4)$
$S = (1, 2)\ (7, 4)\ (21, 4)\ (14, 3)$



$$C(S) = \frac{1}{4} \sum_{\ell=1}^{4} f_\ell = \frac{1}{4}(3 + 15 + 31 + 41) = 22{,}5$$

$(s_1, i_1)\ (s_2, i_2)\ (s_3, i_3)\ (s_4, i_4)$
$S' = (0, 2)\ (3, 1)\ (12, 4)\ (30, 3)$



$$C(S') = \frac{1}{4} \sum_{\ell=1}^{4} f_\ell = \frac{1}{4}(3 + 11 + 27 + 32) = 43{,}5$$

Quindi, una soluzione ottima non può ammettere idle Time

Possiamo restringerci a considerare solo soluzioni senza idle Time

$S = (s_1, i_1)\ (s_2, i_2) \cdots (s_n, i_n)$ è ammissibile per $P(T, 0)$

se

1) $0 = s_\ell$
2) $s_\ell + t_{i_\ell} = s_{\ell+1}$ ✓ $1 \le \ell \le n-1$ (che implica $s_\ell = \sum_{h=1}^{\ell-1} t_{i_h}$)
3) $\{i_1, i_2, \dots, i_n\} = T$ (S contiene tutti e soli i Task in T)

Cioè, i tempi di inizio dei Task nella soluzione sono univocamente determinati dalla posizione dei Task in essa.

Possiamo quindi semplificare la formalizzazione come segue.

Formalizzazione del Problema

$S = i_1\, i_2 \cdots i_n$ ammissibile per $P(T)$

se $\{i_1, i_2, \dots, i_n\} = T$

Costo di S

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(\ell) \qquad \text{con} \quad f(\ell) = \sum_{h=1}^{\ell} t_{i_h}$$

Proprietà di Decomposizione e Sottostruttura Ottima

1) Se $S = i_1\, i_2 \cdots i_n$ è ammissibile per $P(T)$, allora $S' = i_2 \cdots i_n$ è ammissibile per $P(T \setminus \{i_1\})$

2) Se $S' = i_2' i_3' \cdots i_k'$ è ammissibile per $P(T \setminus \{i\})$, allora $S = i \cdot S'$ è ammissibile per $P(T)$

3) Se $S = i_1\, i_2 \cdots i_n$ è ottima per $P(T)$, allora $S' = i_2 \cdots i_n$ è ottima per $P(T \setminus \{i_1\})$

3) Se $S = i_1 i_2 \dots i_n$ è ottima per $P(T)$, allora $S' = i_2 \dots i_n$ è ottima per $P(T, i_1 i_2)$

↳ Dim.

$$C(S') = \frac{1}{n-1} \sum_{\ell=2}^{n} f'(i_\ell) = \frac{1}{n-1} \sum_{\ell=2}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right)$$

$$C(S) = \frac{1}{n} \sum_{\ell=1}^{n} f(i_\ell) = \frac{1}{n} \cdot \sum_{\ell=1}^{n} \left(\sum_{j=1}^{\ell} t_{i_j} \right) =$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + \sum_{\ell=1}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right) \right] = \quad (\text{perché?})$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + \sum_{\ell=2}^{n} \left(\sum_{j=2}^{\ell} t_{i_j} \right) \right] = \quad (\text{perché?})$$

$$= \frac{1}{n} \left[\sum_{\ell=1}^{n} t_{i_1} + (n-1) \cdot C(S') \right] = t_{i_1} + \frac{n-1}{n} C(S')$$

Assumendo ora S ottima per $P(T)$ ma S' non ottima per $P(T \setminus \{i_1\})$, otteniamo che

$\exists\, S'' = i_2' i_3' \dots i_n'$ t.c. $C(S') > C(S'')$

Ma, per proprietà 2) di decomposizione, $S^* = i_1 \circ S'' = i_1 i_2' \dots i_n'$ è ammissibile per $P(T)$ e

$$\begin{array}{ccccc} C(S^*) & = & t_{i_1} + \frac{(n-1)}{n} \cdot C(S'') & & \\ \wedge & & \| & & \| \\ C(S) & = & t_{i_1} + \frac{(n-1)}{n} \cdot & & C(S') \end{array} \wedge$$

$$\Longrightarrow C(S^*) < C(S) \text{ . Assurdo !!}$$

SCELTA GREEDY

La politica di scelta che permette di ottenere sempre una soluzione ottima è la seguente

> Schedulare per primo il Task con la minima durata tra quelli disponibili.

PROPRIETÀ DELLA SCELTA GREEDY

> Dato il problema $P(T)$, con $|T| = n$.
> Sia $i \in T$ il Task con la minima durata tra i Task in T.
> Allora esiste una soluzione ottima $S^* = i_1 i_2 \dots i_n$ t.c. $i_1 = i$

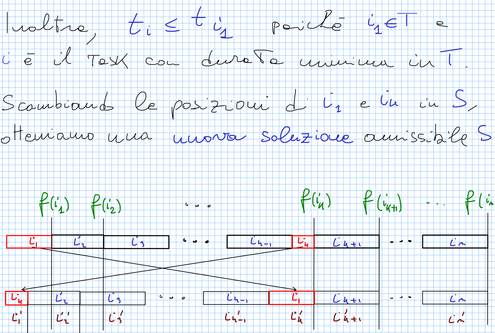
Dim.

Consideriamo una soluzione ottima $S = i_1 i_2 \dots i_n$ per il problema $P(T)$.

Abbiamo 2 possibili casi:

1) $i_1 = i$: in tal caso la Tesi vale banalmente

2) $i_1 \neq i$: una $i \in S$, quindi $i = i_k$ per qualche $2 \leq k \leq n$.
Inoltre, $t_i \leq t_{i_1}$ poiché $i_1 \in T$ e i è il Task con durata minima in T.
Scambiando le posizioni di i_1 e i_k in S, otteniamo una nuova soluzione ammissibile S':



Cioè S' è definibile a partire da S come segue:

$$S' = i_1' i_2' \dots i_n'$$

con $i_1' = i_k$, $i_k' = i_2$ e $i_\ell' = i_\ell$ per ogni $\ell \neq 1$ e $\ell \neq k$

Vediamo ora le seguenti relazioni tra S e S'

a) $\forall\ k \le \ell \le n$ $\quad \sum_{j=1}^{\ell} t_{i'_j} = \sum_{j=1}^{\ell} t_{i_j}$ $\quad \left[f'(i'_\ell) = f(i_\ell) \right]$

Il tempo di completamento dei task dal k-esimo all'n-esimo è lo stesso in S e S'.

Infatti, poiché $i_1 = i'_k$ e $i_k = i'_1$, abbiamo che:

$$f(i'_\ell) = \sum_{j=1}^{\ell} t_{i_j} = t_{i_1} + t_{i_2} + \cdots + t_{i_k} + \cdots + t_{i_\ell} =$$
$$= t_{i'_k} + t_{i'_2} + \cdots + t_{i'_1} + \cdots + t_{i'_\ell} = \sum_{j=1}^{\ell} t_{i'_j} = f'(i'_\ell)$$

b) $\forall\ 1 \le \ell \le k-1$ $\quad \sum_{j=1}^{\ell} t_{i'_j} \le \sum_{j=1}^{\ell} t_{i_j}$ $\quad \left[f'(i'_\ell) \le f(i_\ell) \right]$

Infatti, poiché $i'_1 = i_k$, abbiamo che:

$$f(i'_\ell) = \sum_{j=1}^{\ell} t_{i_j} = t_{i_1} + t_{i_2} + \cdots + t_{i_\ell}$$
$$\qquad\qquad\quad\ \ \| \quad \| \quad\ \ \|$$
$$f'(i'_\ell) = \sum_{j=1}^{\ell} t_{i'_j} = t_{i'_2} + t_{i'_3} + \cdots + t_{i'_\ell} = t_{i_k} + t_{i_2} + \cdots + t_{i_\ell}$$

Ma, per assunzione, $t_{i_k} \le t_{i_1}$, quindi

$$f'(i'_\ell) = \sum_{j=1}^{\ell} t_{i'_j} \le \sum_{j=1}^{\ell} t_{i_j} = f(i'_\ell)$$

Da a) e b) segue immediatamente che

$$C(S') \le C(S)$$

e, quindi, che S' è ottima se S lo è. □

ALGORITMO GREEDY DI SOLUZIONE

La proprietà di scelta greedy ci assicura che una soluzione ottima si ottiene calcolando una permutazione degli n task che ordini i task in maniera crescente (non decrescente).

Il problema si riduce, quindi, al problema dell'ordinamento e il costo in tempo è

$$\Theta(n \log n).$$

PROBLEMA DELLO ZAINO FRAZIONARIO

Siano dati: n oggetti $\{O_1, O_2, \dots O_n\}$. Dell'oggetto O_i se ne ha a disposizione una quantità complessiva di peso P_i. La quantità totale P_i dell'oggetto O_i ha un valore pari a V_i. Sia dato un contenitore (lo zaino) che può sostenere al massimo un peso W (capienza).
Uno **riempimento** dello zaino indica quale porzione di ciascun oggetto inserire nello zaino, senza eccedere la sua capacità.
Calcolare il riempimento di valore massimo per lo zaino.
(valore dello zaino è inteso come la somma dei valori delle porzioni degli oggetti in esso contenute)

Decomposizione:
$P(O, w)$ → capacità residua dello zaino ($w \leq W$)
→ insieme di oggetti a disposizione ($O \subseteq O$)

$S = (i_1, q_1) \cdots (i_k, q_k)$ è ammissibile per $P(O, w)$ se
- $i_1, \dots, i_k \subseteq O$
- $0 < q_\ell \leq P_{i_\ell} \quad \forall\, 1 \leq \ell \leq k$
- $\sum_{\ell=1}^{k} q_\ell \leq w$

$$C(S) = \sum_{\ell=1}^{k} \frac{q_\ell}{P_{i_\ell}} \cdot v_{i_\ell}$$

$\frac{q_\ell}{P_{i_\ell}}$ è la frazione di peso dell'oggetto O_{i_ℓ} contenuta in S

$$C(S) = \sum_{\ell=1}^{k} q_\ell \cdot \frac{v_{i_\ell}}{P_{i_\ell}}$$

$\frac{v_{i_\ell}}{P_{i_\ell}}$ è il valore di una unità di peso dell'oggetto O_{i_ℓ}

SOTTOSTRUTTURA OTTIMA

Se $(i_1, q_1) \cdots (i_k, q_k)$ è sol ottima per $P(O, w)$
allora $(i_1, q_1) \cdots (i_{k-1}, q_{k-1})$ è sol ottima per $P(O \setminus \{i_k\}, w - q_k)$

SCELTA GREEDY

Per ogni sottoproblema $P(O, w)$ scegliere (i_ℓ, q_ℓ) tale che:
1) i_ℓ è l'oggetto col massimo valore per unità di peso tra gli oggetti di O
i_ℓ è t.c. $\frac{v_{i_\ell}}{P_{i_\ell}} \geq \frac{v_i}{P_i} \quad \forall\, i \in O$
2) q_ℓ è la massima quantità di i_ℓ che può essere inserita nello zaino
$q_\ell = \min(P_{i_\ell}, w)$

PROPRIETÀ DI SCELTA GREEDY

Una soluzione ottima al problema $P(O, w)$ contiene la scelta greedy

Dim Sia $S = (i_1, q_1) \cdots (i_n, q_n)$ una soluzione ottima per $P(O, w)$ e sia J l'oggetto col massimo valore per unità di peso. Cioè:

$$\frac{v_J}{P_J} \geq \frac{v_i}{P_i} \quad \forall\, i \in O$$

Abbiamo 2 casi:

a) J è contenuto in S.
Quindi esiste un $1 \leq \ell \leq k$ tale che $(i_\ell, q_\ell) \in S$ e $i_\ell = J$
Sappiamo che $q_\ell \leq P_\ell$ per ammissibilità di S.
Abbiamo altri 2 sotto-casi:
1) $q_\ell = P_\ell$ o $q_\ell = w$.
In entrambi i casi (i_ℓ, q_ℓ) è la scelta greedy e la tesi è dimostrata
2) $q_\ell < P_\ell$ e $P_\ell < w$, allora prendiamo un qualsiasi indice $\pi \neq \ell$, con $1 \leq \pi \leq k$.
Sia (i_π, q_π) la π-esima scelta in S
e poniamo $x = \min(q_\pi, P_\ell - q_\ell)$.

π esiste per forza se S è ottima e $P_\ell < w$

Se sostituiamo in S
(J, q_J) con $(J, q_J + x)$
(i_π, q_π) con $(i_\pi, q_\pi - x)$
otteniamo una nuova soluzione S' con lo stesso peso di S.
Il costo di S' è:
$$C(S') = C(S) - \frac{v_\pi}{P_\pi} \cdot x + \frac{v_J}{P_J} \cdot x$$
Ma $\frac{v_\pi}{P_\pi} \leq \frac{v_J}{P_J} \Rightarrow C(S') \geq C(S)$.
Se $(J, q_J + x)$ non è ancora la scelta ottima (cioè se $q_J + x < P_J$ e $q_J + x < w$) ripetiamo lo stesso ragionamento con un altro oggetto in S, fino a ottenere una soluzione con la scelta greedy $\Rightarrow$ Tesi

b) J non è contenuto in S.
Sia $1 \leq \pi \leq k$ e (i_π, q_π) la π-esima scelta in S e $x = \min(q_\pi, P_J)$
Sostituendo in S
(i_π, q_π) con $(i_\pi, q_\pi - x)$ [$x = q_\pi - x > 0$]
e aggiungendo
(J, x)
otteniamo una soluzione ammissibile (il peso è lo stesso) e il costo è tale che
$$C(S') \geq C(S)$$
per lo stesso ragionamento usato nel caso 1.b)
Se (J, x) non è la scelta greedy (cioè se $x < w$ e $x < P_J$) possiamo applicare il caso 1.b) ripetutamente fino a ottenerla.
La tesi vale quindi anche in questo caso.

OSSERVAZIONE: La strategia greedy non garantisce l'ottimo per il problema originario dello ZAINO 0-1.

CONTROESEMPIO

$W = 4 \qquad O = \{1, 2, 3\}$

$P_1 = 3 \qquad v_1 = 5$
$P_2 = P_3 = 2 \qquad v_2 = v_3 = 3$

$\frac{v_1}{P_1} = \frac{5}{3} \qquad \frac{v_2}{P_2} = \frac{v_3}{P_3} = \frac{3}{2} \qquad \left[\frac{5}{3} > \frac{3}{2}\right]$

SOLUZIONE GREEDY: $S = \{1\}$ $\qquad C(S) = 5$
SOLUZIONE OTTIMA: $S^* = \{2, 3\}$ $\qquad C(S^*) = 6$

RETI DI FLUSSO

PROBLEMA: UNA RETE DI FLUSSO È COSTITUITA DA:

- UN GRAFO ORIENTATO $G=(V,E)$
- PER OGNI ARCO $e=(u,v)\in E$, $c(e)\in \mathbb{N}$ DETTO CAPACITÀ DI e
- UN VERTICE SORGENTE s E UN VERTICE DESTINAZIONE t TALI CHE s NON HA ARCHI ENTRANTI E t NON HA ARCHI USCENTI

UN **FLUSSO DA s A t** IN G È UNA FUNZIONE

$$f: E \to \mathbb{R}^+ \qquad (\mathbb{R}^+ : \text{reali} \geq 0)$$

FLUSSO ENTRANTE IN v: $f^{IN}(v) = \sum_{\substack{e=(u,v)\\ e\in E}} f(e)$

FLUSSO USCENTE DA v: $f^{OUT}(v) = \sum_{\substack{e=(v,u)\\ e\in E}} f(e)$

TALE CHE

- CAPACITÀ: a) $0 \leq f(e) \leq c(e)$ PER OGNI ARCO $e\in E$
- CONSERVAZIONE: b) $f^{IN}(v) = f^{OUT}(v)$ PER OGNI VERTICE $v \in V \setminus \{s,t\}$
- c) $f^{IN}(s) = 0$ E $f^{OUT}(t) = 0$

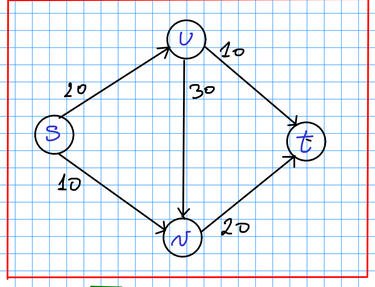
OBIETTIVO:

CALCOLARE UN **FLUSSO MASSIMO** DA s A t IN G

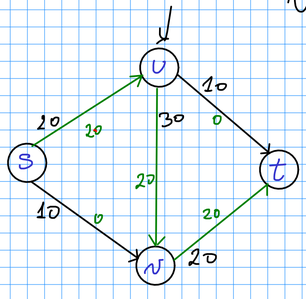
UN FLUSSO f T.C. $\text{VAL}(f) = f^{OUT}(s)$ SIA MASSIMO.

VALORE DEL FLUSSO $= f^{IN}(t)$

ESEMPIO:



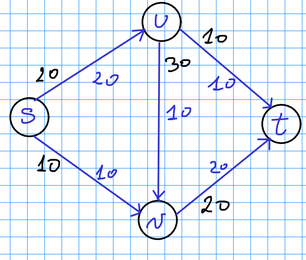
CAPACITÀ:
- $c(s,u) = 20$
- $c(u,v) = 30$
- $c(v,t) = 20$
- $c(s,v) = 10$
- $c(u,t) = 10$



$\text{Val}(f) = 20$

- $f(s,u) = 20$
- $f(u,v) = 20$
- $f(v,t) = 20$
- $f(s,v) = 0$
- $f(u,t) = 0$

FLUSSO NON OTTIMO



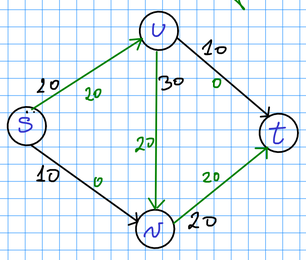
- $f'(s,u) = 20$
- $f'(u,v) = 10$
- $f'(v,t) = 20$
- $f'(s,v) = 10$
- $f'(u,t) = 10$

FLUSSO OTTIMO

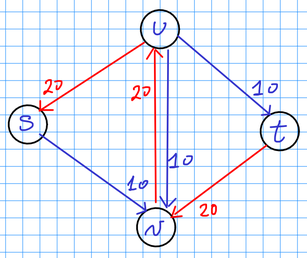
$\text{Val}(f') = 30$

GRAFO RESIDUO

FLUSSO f



$G_f^R = (V, E_f^R)$ GRAFO RESIDUO PER f



$\forall (u,v) \in E$:

- $f(u,v) < c(u,v) \Longrightarrow (u,v) \in E_f^R$ con $c_f^R(u,v) = c(u,v) - f(u,v)$

ARCO IN AVANTI

flusso residuo disponibile da u a v

- $f(u,v) > 0 \Longrightarrow (v,u) \in E_f^R$ con $c_f^R(v,u) = f(u,v)$

ARCO A RITROSO

flusso che può essere spinto indietro da v ad u (a ritroso lungo (u,v))

IL GRAFO RESIDUO CI DICE QUANTO FLUSSO PUÒ ANCORA ESSERE INVIATO VERSO t E QUANTO FLUSSO PUÒ ESSERE RIMANDATO INDIETRO VERSO s

IDEA: OGNI PERCORSO π DA s A t NEL GRAFO RESIDUO G_f^R INDUCE UN NUOVO FLUSSO f_π TALE CHE

$$VAL(f_\pi) > VAL(f)$$

ESEMPIO: CONSIDERIAMO IL PERCORSO $\pi = s\,v\,u\,t$ in G_f^R.

LA CAPACITÀ MINIMA DI UN ARCO DI π È

$$c_{MIN} = c_f^R(s,v) = c_f^R(u,t) = 10$$

FLUSSO AUMENTATO

DEFINIAMO f_π A PARTIRE DA f NEL SEGUENTE MODO:

$f_\pi(x,y) = f(x,y)$ se $(x,y) \notin \pi$

$f_\pi(x,y) = f(x,y) + c_{MIN}$ se $(x,y) \in \pi$ E (x,y) È ARCO IN AVANTI

$f_\pi(x,y) = f(x,y) - c_{MIN}$ se $(y,x) \in \pi$ E (y,x) È ARCO A RITROSO

$f(u,v) = 20$

$c_{MIN} = 10$

$f_\pi(s,u) = 20$

$f_\pi(u,v) = 20 - 10 = 10$

$f_\pi(v,t) = 10$

$f_\pi(s,v) = 0 + 10 = 10$

$f_\pi(u,t) = 0 + 10 = 10$

$f(s,v) = 0$

$f(u,t) = 0$

IN G_f^R:

(v,u) È ARCO A RITROSO

(s,v) E (u,t) SONO ARCHI IN AVANTI

NOTARE CHE f_π È PROPRIO f' NELL'ESEMPIO PRECEDENTE QUINDI f_π IN QUESTO CASO È ANCHE OTTIMO.

TEOREMA: SIA DATA UNA RETE DI FLUSSO $G=(V,E)$, UN FLUSSO f SU G E SIA G_f^R IL GRAFO RESIDUO DI G RISPETTO A f. PER OGNI PERCORSO π IN G_f^R, IL FLUSSO AUMENTATO f_π HA LA SEGUENTE PROPRIETÀ:

f_π È UN FLUSSO SU G

DIM: DOBBIAMO DIMOSTRARE CHE VALGONO LE SEGUENTI 3 PROPRIETÀ:

a) **CAPACITÀ:** $0 \le f_\pi(e) \le c(e)$, PER OGNI $e \in E$

- Se $e \in \pi$ è in avanti allora $f_\pi(e) = f(e) + c_{MIN}$

  ma $c_{MIN} \le c_f(e) = c(e) - f(e) \Rightarrow f_\pi(e) \le f(e) + c(e) - f(e) = c(e)$

- Se $\bar{e} \in \pi$ è a ritroso allora $f_\pi(e) = f(e) - c_{MIN}$ (con $\bar{e}$ l'inverso di e)

  ma $c_{MIN} \le c_f(\bar{e}) = f(e) \Rightarrow f_\pi(e) \ge f(e) - f(e) = 0$

- Per gli altri archi di E, $f(e) = f_\pi(e) \Rightarrow$ la conclusione vale.

b) **CONSERVAZIONE:** $f_\pi^{IN}(v) = f_\pi^{OUT}(v)$, PER OGNI $v \in V \setminus \{s,t\}$

- Se $v \notin \pi$ la Tesi è ovvia, poiché nessun arco entrante o uscente da v viene aumentato $\Rightarrow f_\pi^{IN}(v) = f^{IN}(v) = f^{OUT}(v) = f_\pi^{OUT}(v)$

- Se $v \in \pi$, invece, esattamente 1 arco entrante in v ((k,v)) e uno uscente da v ((v,z)) del grafo residuo verranno aumentati, poiché π è semplice e contiene una sola occorrenza di v.

  Ci sono 4 casi possibili:

  1) (k,v) e (v,z) sono entrambi in avanti
  2) (k,v) e (v,z) sono entrambi a ritroso
  3) (k,v) è in avanti e (v,z) è a ritroso
  4) (k,v) è a ritroso e (v,z) è in avanti

CASO 1:

$$f_\pi((k,v)) = f((k,v)) + c_{MIN}$$
$$f_\pi((v,z)) = f((v,z)) + c_{MIN}$$

Quindi $f_\pi^{IN}(v) = f^{IN}(v) + c_{MIN}$ [grazie all'aumento di (k,v)]

$f_\pi^{OUT}(v) = f^{OUT}(v) + c_{MIN}$ [grazie all'aumento di (v,z)]

Segue che $f_\pi^{IN}(v) = f_\pi^{OUT}(v)$

CASO 2: Simmetrico di CASO 1.

CASO 3:

$f_\pi((k,v)) = f((k,v)) + c_{MIN}$ poiché (k,v) è in avanti

$f_\pi((z,v)) = f((z,v)) - c_{MIN}$ poiché (v,z) è a ritroso

Quindi $f_\pi^{OUT}(v) = f^{OUT}(v)$ [nessun arco uscente da v cambia]

$f_\pi^{IN}(v) = f^{IN}(v) + c_{MIN} - c_{MIN} = f^{IN}(v)$ [il flusso c_{MIN} viene sottratto da (z,v) e aggiunto a (k,v)]

CASO 4: Simmetrico di CASO 3.

c) $f_\pi^{IN}(s) = 0$ E $f_\pi^{OUT}(t) = 0$. Infatti:

- π parte da s e arriva in t ed è semplice $\Rightarrow$
- π non contiene archi entranti in s o uscenti da t
- Quindi $f_\pi^{IN}(s) = f^{IN}(s)$ e $f_\pi^{OUT}(t) = f^{OUT}(t)$.

TEOREMA: SIA DATA UNA RETE DI FLUSSO $G=(V,E)$, UN FLUSSO f SU G E SIA G_f^R IL GRAFO RESIDUO DI G RISPETTO A f. PER OGNI PERCORSO π IN G_f^R, IL FLUSSO AUMENTATO f_π HA LA SEGUENTE PROPRIETÀ:

$$VAL(f_\pi) > VAL(f)$$

DIM: E' SUFFICIENTE FAR VEDERE CHE $f_\pi^{OUT}(s) = f^{OUT}(s) + c_{MIN}$ E $c_{MIN} > 0$

- CHE $c_{MIN} > 0$ DISCENDE DAL FATTO CHE c_{MIN} È LA CAPACITÀ $c_f(e)$ DI UN QUALCHE ARCO DI π NEL GRAFO RESIDUO G_f^R E, PER COSTRUZIONE DI G_f^R, TUTTI I SUOI ARCHI HANNO CAPACITÀ POSITIVA.
- POICHÉ π È UN PERCORSO DA s A t IN G_f^R, π CONTIENE UN QUALCHE ARCO $e=(s,z) \in E_f^R$. L'ARCO e È NECESSARIAMENTE UN ARCO IN AVANTI, POICHÉ NON ESISTONO IN G ARCHI ENTRANTI IN s.

  MA ALLORA $f_\pi(e) = f(e) + c_{MIN}$ E, QUINDI,

$$val(f_\pi) = f_\pi^{OUT}(s) = f^{OUT}(s) + c_{MIN} > f^{OUT}(s) = val(f)$$

L'ALGORITMO DI FORD-FULKERSON

```
MAX-FLOW (G, s, t)
    FOR e ∈ E DO
        f(e) = 0
    G_f^R = GRAFO_RESIDUO(G, f)
    π = SEARCH-PATH(G_f^R, s, t)
    WHILE π ≠ NIL DO
        c_MIN = min{ c_f(e) | e ∈ π }
        f = AUMENTA(f, c_MIN, π)
        G_f^R = UPDATE(G_f^R, f)
        π = SEARCH-PATH(G_f^R, s, t)
```

AD ESEMPIO TRAMITE
- DFS
- BFS

CALCOLA IL FLUSSO AUMENTATO f_π

GENERA IL NUOVO GRAFO RESIDUO

GRAFO_RESIDUO, SEARCH-PATH, AUMENTA E UPDATE POSSONO TUTTE ESSERE IMPLEMENTATE IN MODO CHE RICHIEDANO TEMPO $O(|G|)$ LINEARE SULLA DIMENSIONE DEL GRAFO IN INGRESSO

LA TERMINAZIONE E IL TEMPO DI ESECUZIONE DI MAX-FLOW DIPENDONO, QUINDI, DAL NUMERO DI ITERAZIONI DEL CICLO WHILE.

TEOREMA: DATA UNA RETE DI FLUSSO $G=(V,E)$ CON CAPACITÀ A VALORI INTERI: $c(e) \in \mathbb{N}$, PER OGNI $e \in E$.
L'ALGORITMO DI FORD-FULKERSON ESEGUITO SU G TERMINA DOPO AL MASSIMO C ITERAZIONI DEL WHILE

DOVE $$C = \sum_{\substack{e \in E \\ e=(s,v)}} c(e)$$ ← MASSIMO FLUSSO USCENTE DA s

DIM: Abbiamo già osservato che se f_π è il flusso aumentato di f allora $VAL(f_\pi) > VAL(f)$

Ovviamente, se f^* è un flusso massimo,

$$VAL(f^*) \leq C \qquad \left[VAL(f^*) = \sum_{\substack{e \in E \\ e=(s,v)}} f^*(e) \leq \sum_{\substack{e \in E \\ e=(s,v)}} c(e) = C \right]$$

Ci basta dimostrare che ad ogni iterazione del ciclo WHILE il flusso aumenta il suo valore di almeno 1 unità. Da questo la tesi discende immediatamente.

A tal fine, possiamo osservare che se le capacità sono intere allora ad ogni iterazione vale che

$$\forall e \in E_f^R, \quad c_f(e) \in \mathbb{N}$$

(le capacità residue in G_f^R sono valori interi ≥ 0)

CASO BASE Inizialmente il flusso è nullo per ogni arco e quindi la tesi vale all'inizio della I^a iterazione.

CASO INDUTTIVO Assumiamo per induzione che all'inizio di una arbitraria iterazione il flusso f abbia valori interi.
Allora le capacità residue in G_f^R sono intere.
Sia π un percorso da s a t in G_f^R, la minima capacità residua di un arco di π è c_{MIN} un intero > 0
Quindi, $c_{MIN} \geq 1$ e intero.
Ma allora il flusso aumentato f_π è anch'esso a valori interi (somma/sottrae c_{MIN} dai valori di f)

Poiché $c_{MIN} \geq 1$, abbiamo anche che

$$VAL(f_\pi) \geq VAL(f) + 1$$

Quindi, ad ogni iterazione il flusso aumenta il suo valore di almeno una unità.

Dopo al più C iterazioni il valore del flusso f avrà raggiunto il suo estremo superiore C.
Inoltre, se ciò accade, nel grafo residuo G^R non può contenere archi in uscenti da s, quindi non esiste un percorso da s a t e l'algoritmo termina.

L'ALGORITMO DI FORD-FULKERSON OTTIENE IL FLUSSO MASSIMO DI UNA RETE DI FLUSSO G IN TEMPO $O(|G| \cdot C)$

CON $$C = \sum_{\substack{e \in E \\ e=(s,v)}} c(e)$$

FLUSSO MASSIMO E MINIMO TAGLIO

DATA UNA RETE DI FLUSSO $G=(V,E)$ CON FUNZIONE DI CAPACITA' $c: E \to \mathbb{N}$, SORGENTE $s \in V$ E DESTINAZIONE $t \in V$, UN **TAGLIO DI G** E' UNA COPPIA DI INSIEMI (A,B) TALE CHE

1) A E B FORMANO UNA PARTIZIONE DI V
   - $A \cap B = \emptyset$
   - $A \cup B = V$
2) $s \in A$ E $t \in B$

LA CAPACITA' DI UN TAGLIO (A,B) E' DEFINITA COME LA SOMMA DELLE CAPACITA' DEGLI ARCHI USCENTI DA A:

$$c(A,B) = \sum_{\substack{(u,v)\in E \\ u\in A \\ v\in B}} c(u,v)$$

DATO UN FLUSSO f DA s A t E UN TAGLIO (A,B) DEFINIAMO:

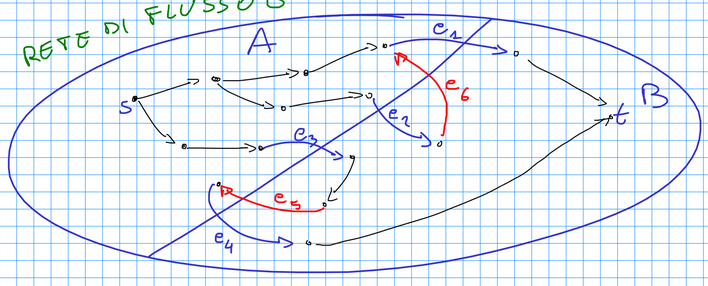
$$f^{out}(A) = \sum_{\substack{(u,v)\in E \\ u\in A \\ v\in B}} f(u,v)$$

QUANTITA' DI FLUSSO CHE ESCE DA A E VA IN B

$$f^{in}(A) = \sum_{\substack{(u,v)\in E \\ u\in B \\ v\in A}} f(u,v)$$

QUANTITA' DI FLUSSO CHE DA B TORNA IN A

RETE DI FLUSSO G



$$f^{out}(A) = f(e_1)+f(e_2)+f(e_3)+f(e_4)$$

$$f^{in}(A) = f(e_5)+f(e_6)$$

LA RELAZIONE TRA FLUSSI E TAGLI E' DATA DAL SEGUENTE TEOREMA

TEOREMA: DATO UN FLUSSO f DA s A t E UN TAGLIO (A,B) ARBITRARIO:

$$val(f) = f^{out}(A) - f^{in}(A)$$

DIM.:

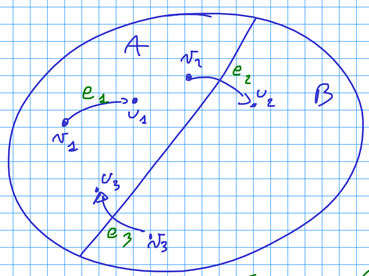
$$val(f) = \sum_{(s,u)\in E} f(s,u) = f^{out}(s) = f^{out}(s) - f^{in}(s)$$ (ricordate che $f^{in}(s)=0$)

$$\forall v \in A\setminus\{s\} \quad \left(f^{out}(v) - f^{in}(v)\right) = 0 \implies val(f) = \underbrace{\sum_{v\in A}\left(f^{out}(v) - f^{in}(v)\right)}_{f(A,B)}$$

(CONSERVAZIONE DEL FLUSSO)

$$f^{out}(v) - f^{in}(v) = \sum_{\substack{e=(v,u)\\ e\in E}} f(e) - \sum_{\substack{e=(u,v)\\ e\in E}} f(e)$$

$$f^{out}(u) - f^{in}(u) = \sum_{\substack{e=(u,v)\\ e\in E}} f(e) - \sum_{\substack{e=(v,u)\\ e\in E}} f(e)$$



RETE DI FLUSSO G

- Ogni arco $e_1=(v_1,u_1)$ i cui estremi sono entrambi in A contribuisce col segno + a $f^{OUT}(v_1)$ e con il segno − a $f^{IN}(u_1)$. Quindi i suoi 2 contributi si annullano in $f(A,B)$.
- Ogni arco $e_2=(v_2,u_2)$ con $v_2\in A$ e $u_2\in B$ contribuisce solo col segno + a $f(A,B)$.
- Ogni arco $e_3=(v_3,u_3)$ con $v_3\in B$ e $u_3\in A$ contribuisce solo col segno − a $f(A,B)$.

In altre parole, gli unici archi che danno un contributo non nullo sono gli archi che entrano in A (contributo −) e quelli che escono da A (contributo +)

Quindi $f(A,B)$ si può riscrivere:

$$f(A,B)=\sum_{v\in A}\left(f^{OUT}(v)-f^{IN}(v)\right)=\sum_{\substack{(v,u)\in E\\ v\in A\\ u\in B}} f(v,u)-\sum_{\substack{(u,v)\in E\\ u\in B\\ v\in A}} f(u,v)=f^{OUT}(A)-f^{IN}(A)$$

Da cui otteniamo la tesi: $VAL(f)=f^{OUT}(A)-f^{IN}(A)$

E' IMMEDIATO OSSERVARE CHE PER OGNI TAGLIO (A,B) VALE

$$f^{OUT}(A)=f^{IN}(B) \quad \text{e} \quad f^{IN}(A)=f^{OUT}(B)$$

QUINDI

$$VAL(f)=f^{OUT}(A)-f^{IN}(A)=f^{OUT}(B)-f^{IN}(B)$$

E QUINDI

$$VAL(f)=f^{OUT}(t)-f^{IN}(t)=f^{IN}(t)$$

Prendendo $(V\setminus\{t\},\{t\})$ come taglio e ricordando che $f^{OUT}(t)=0$

POSSIAMO ORA OTTENERE UN LIMITE SUPERIORE AL VALORE DEI FLUSSI:

TEOREMA: DATO UN FLUSSO f, QUALSIASI TAGLIO (A,B) SODDISFA:

$$VAL(f)\le C(A,B)$$

DIM: Sappiamo che $VAL(f)=f^{OUT}(A)-f^{IN}(A)$

Poiché $f^{IN}(A)\ge 0$, segue che

$$VAL(f)\le f^{OUT}(A)=\sum_{\substack{e=(v,u)\in E\\ v\in A\\ u\in B}} f(e)$$

Ma $f(e)\le c(e)$ per qualsiasi arco $e\in E$, quindi

$$VAL(f)\le f^{OUT}(A)=\sum_{\substack{e=(v,u)\in E\\ v\in A\\ u\in B}} f(e)\le \sum_{\substack{e=(v,u)\in E\\ v\in A\\ u\in B}} c(e)=C(A,B)$$

QUESTA PROPRIETÀ CI DICE CHE NESSUN FLUSSO PUÒ AVERE VALORE MAGGIORE DEL TAGLIO DI CAPACITÀ MINIMA.

TEOREMA: SIA f UN FLUSSO TALE CHE IN G_f^R NON ESISTE UN PERCORSO DA s A t. ALLORA ESISTE UN TAGLIO (A^*, B^*) TALE CHE

$$VAL(f) = C(A^*, B^*)$$

DIM: È sufficiente esibire un Taglio con la proprietà desiderata. Definiamo, quindi, il Taglio nel seguente modo:

$$A^* = \{ v \in V \mid s \rightsquigarrow v \text{ in } G_f^R \}$$ (v è raggiungibile da s nel grafo residuo)

$$B^* = V \setminus A^*$$

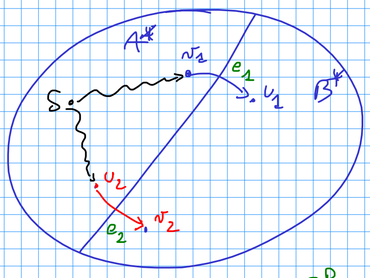
Chiaramente (A^*, B^*) è una partizione di V, $s \in A^*$ (s è raggiungibile da sé stesso in G_f^R) e $t \in B^*$ (t non è raggiungibile da s in G_f^R per ipotesi del Teorema). Quindi (A^*, B^*) è effettivamente un Taglio.

Consideriamo gli archi della rete G che attraversano il Taglio (A^*, B^*).

CASO 1: $(v_1, u_1) \in E$ con $v_1 \in A^*$ e $u_1 \in B^*$

Si dimostra che $f(v_1, u_1) = c(v_1, u_1)$.

Infatti se $f(v_1, u_1) < c(v_1, u_1)$ l'arco $(v_1, u_1) \in E_f^R$ e u_1 sarebbe raggiungibile da s in G_f^R, il che contraddice l'ipotesi che $u_1 \in B^*$



GRAFO RESIDUO G_f^R (entrambi gli archi sono impossibili)

CASO 2: $(v_2, u_2) \in E$ con $v_2 \in B^*$ e $u_2 \in A^*$

Si dimostra che $f(v_2, u_2) = 0$.

Infatti, se così non fosse, in G_f^R esisterebbe l'arco (u_2, v_2) con $c_f(u_2, v_2) = f(v_2, u_2) > 0$. Ma allora v_2 sarebbe raggiungibile da s in G_f^R, contraddicendo l'ipotesi che $v_2 \in B^*$.

Quindi, ogni arco che passa da A^* a B^* ha flusso pari alla sua capacità e ogni arco che da B^* torna in A^* ha flusso nullo. Questo significa che

$$f^{OUT}(A^*) = \sum_{\substack{e=(v,u)\in E \\ v\in A^* \\ u\in B^*}} f(e) = \sum_{\substack{e=(v,u)\in E \\ v\in A^* \\ u\in B^*}} c(e)$$

$$f^{IN}(A^*) = \sum_{\substack{e=(v,u)\in E \\ v\in B^* \\ u\in A^*}} f(e) = 0$$

Quindi

$$VAL(f) = f^{OUT}(A^*) - f^{IN}(A^*) = \sum_{\substack{e=(v,u)\in E \\ v\in A^* \\ u\in B^*}} c(e) - 0 = C(A^*, B^*)$$

SE CONSIDERIAMO IL FLUSSO $\bar{f}$ RESTITUITO DA FORD-FULKERSON, NON ESISTE UN PERCORSO DA s A t IN $G_{\bar{f}}^R$. DAL TEOREMA PRECEDENTE OTTENIAMO CHE IL TAGLIO (A^*, B^*) INDOTTO SODDISFA $VAL(\bar{f}) = C(A^*, B^*)$. POICHÉ OGNI FLUSSO f SODDISFA $VAL(f) \leq C(A^*, B^*)$ [DA UNO DEI TEOREMI PRECEDENTI], OTTENIAMO CHE

$\bar{f}$ È UN FLUSSO MASSIMO DI G

LA DIMOSTRAZIONE DEL TEOREMA PRECEDENTE CI PERMETTE DI OTTENERE UN ALGORITMO CHE CALCOLA IL MINIMO TAGLIO DI G IN TEMPO $O(|G| \cdot C)$, CON C LA SOMMA DELLE CAPACITÀ DEGLI ARCHI USCENTI DA s:

1) ESEGUIRE FORD-FULKERSON SU G PER OTTENERE IL FLUSSO MASSIMO f;
2) DERIVARE DAL GRAFO RESIDUO DI f IL TAGLIO MINIMO (A^*, B^*).

DALLE OSSERVAZIONI PRECEDENTI DISCENDE CHE:

TEOREMA [MAX-FLOW/MIN-CUT]: IN OGNI RETE DI FLUSSO G, IL VALORE DEL FLUSSO MASSIMO DA s A t È UGUALE ALLA MINIMA CAPACITÀ DI UN TAGLIO s-t.

APPLICAZIONI

CONSIDERIAMO 2 PROBLEMI:

- MATCHING BIPARTITO IN GRAFI NON ORIENTATI
- SEGMENTAZIONE DI IMMAGINI

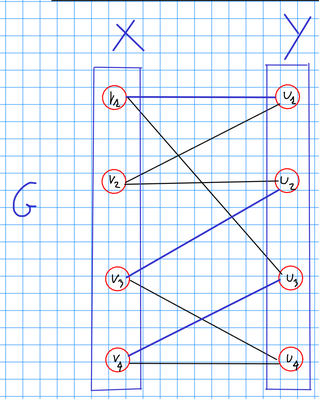
QUESTI PROBLEMI SONO FACILMENTE RIDUCIBILI A MAX-FLOW O MIN-CUT E, QUINDI, RISOLVIBILI TRAMITE ALGORITMO DI FORD-FULKERSON.

MATCHING BIPARTITO

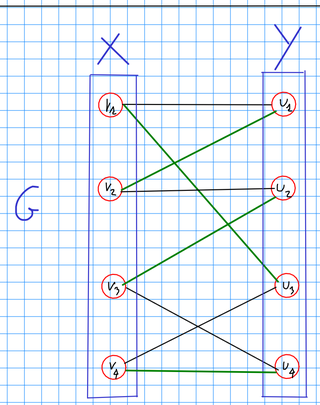
SIA DATO UN GRAFO $G=(V,E)$ NON ORIENTATO E BIPARTITO, CIOÈ TALE CHE V PUÒ ESSERE PARTIZIONATO IN $V = X \cup Y$ [CON $X \cap Y = \emptyset$] E PER OGNI ARCO $(u,v) \in E$, $u \in X$ E $v \in Y$ (NESSUN ARCO CONNETTE VERTICI DELLO STESSO INSIEME NELLA PARTIZIONE)

UN MATCHING IN G È UN SOTTOINSIEME $M \subseteq E$ TALE CHE NESSUN VERTICE DI X O DI Y COMPARE IN PIÙ DI UN ARCO DI M CIOÈ: SE $(u,v) \in M$ E $(z,k) \in M$ ALLORA $\{u,v\} \cap \{z,k\} = \emptyset$

UN MATCHING M IN G SI DICE PERFETTO SE OGNI VERTICE DI X E OGNI VERTICE DI Y COMPARE IN UNA QUALCHE COPPIA IN M.



$M = \{(v_1,u_1), (v_3,u_2), (v_4,u_3)\}$
MATCHING NON MASSIMO (E NON PERFETTO)



$M' = \{(v_1,u_3), (v_2,u_1), (v_3,u_2), (v_4,u_4)\}$
MATCHING MASSIMO (E PERFETTO)

PROBLEMA: DATO UN GRAFO BIPARTITO G, CALCOLARE UN MATCHING DI TAGLIA MASSIMA PER G.

L'IDEA È QUELLA DI COSTRUIRE, A PARTIRE DA G, UNA ISTANZA DEL PROBLEMA DI **MAX-FLOW** PER UNA RETE DI FLUSSO G' OTTENUTA NEL SEGUENTE MODO:

DATO $G=(V,E)$ CON (X,Y) LA BIPARTIZIONE DI V

DEFINIAMO $G'=(V',E')$ IL GRAFO ORIENTATO TALE CHE

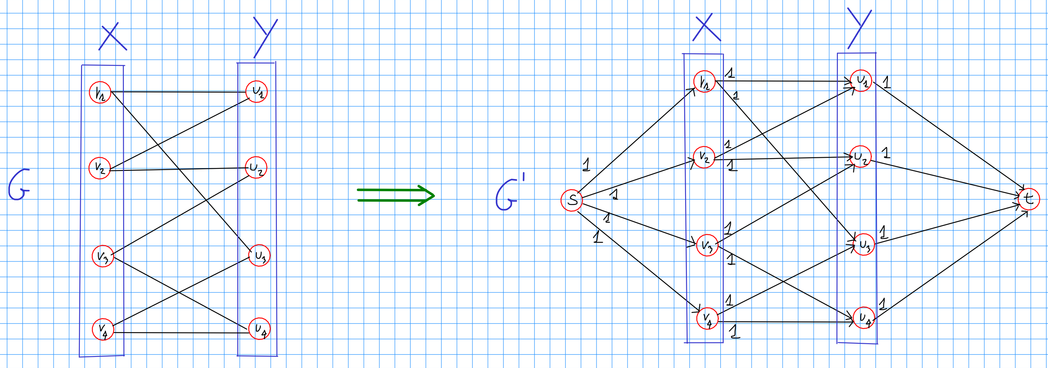
$$V' = V \cup \{s,t\}$$

$$E' = \{(x,y)\in E \mid x\in X \in y\in Y\} \cup \{(s,x) \mid x\in X\} \cup \{(y,t) \mid y\in Y\}$$

E' CONTIENE:
- UN ARCO ORIENTATO, DA X A Y, PER OGNI ARCO DI G
- UN ARCO DA s A OGNI VERTICE DI X
- UN ARCO DA OGNI VERTICE DI Y A t

OGNI ARCO DI G' HA CAPACITÀ 1: $c(v,u)=1 \quad \forall (v,u)\in E'$

LA COPPIA (G',c) È QUINDI UNA **RETE DI FLUSSO**.



TEOREMA: PER OGNI **FLUSSO** A VALORI INTERI f IN G' DI VALORE k ESISTE UN **MATCHING** M_f IN G DI TAGLIA k.

DIM.: PER IPOTESI f HA VALORI INTERI, QUINDI $f(v,u)\in\mathbb{N}$ PER OGNI $(v,u)\in E$. PER DEFINIZIONE DI G' AVREMO ALLORA CHE $f(v,u)\in\{0,1\}$ PER OGNI $(v,u)\in E'$.

DEFINIAMO ORA M_f COME SEGUE

$$M_f = \{(v,u) \mid (v,u)\in E \in f(v,u)=1\}$$

POSSIAMO DIMOSTRARE LE SEGUENTI:

A) OGNI VERTICE DI X E DI Y COMPARE IN AL PIÙ UNA COPPIA DI M_f;

B) M_f CONTIENE ESATTAMENTE k ARCHI.

A) SUPPONIAMO, PER ASSURDO, CHE ESISTANO 2 ARCHI DI M_f USCENTI DALLO STESSO NODO $x\in X$. POICHÉ f HA VALORI INTERI E OGNI ARCO HA CAPACITÀ 1, ENTRAMBI GLI ARCHI HANNO FLUSSO 1. IL FLUSSO USCENTE DA x, $f^{OUT}(x)$, SAREBBE ALLORA ≥ 2. PER PROPRIETÀ DI **CONSERVAZIONE DEI FLUSSI**, $f^{IN}(x) = f^{OUT}(x) \geq 2$. MA x IN G' HA SOLO UN ARCO ENTRANTE DA s CON CAPACITÀ 1, IL CHE È **ASSURDO**. UN RAGIONAMENTO ANALOGO VALE PER I NODI DI Y.

B) POICHÉ f HA VALORI INTERI E LE CAPACITÀ DEGLI ARCHI DI G' SONO TUTTE 1, OGNI ARCO $e\in M_f$ HA FLUSSO $f(e)=1$ E TUTTI GLI ARCHI $e'\notin M_f$ HANNO FLUSSO $f(e')=0$.

POICHÉ $VAL(f)=k$, ESATTAMENTE k VERTICI DI X RICEVERANNO UN FLUSSO UNITARIO E, PER VINCOLO DI CONSERVAZIONE, DA ESSI PUÒ USCIRE SOLO UNA UNITÀ DI FLUSSO LUNGO UNO SOLO DEI SUOI ARCHI. QUINDI, ESATTAMENTE k ARCHI DA X A Y AVRANNO FLUSSO 1, DA CUI $|M_f|=k$.

TEOREMA: PER OGNI MATCHING M IN G DI TAGLIA K, ESISTE UN FLUSSO f_M IN G' DI VALORE K.

DIM.: IL FLUSSO f_M DESIDERATO ASSEGNA FLUSSO 1 IN G' AD OGNI ARCO $(x,y) \in M$, A (s,x) E A (y,t). OTTENIAMO COSÌ UN FLUSSO DA s A t DI VALORE $val(f_M) = K$. È IMMEDIATO VERIFICARE CHE f_M SODDISFA SIA IL VINCOLO DI CAPACITÀ CHE QUELLO DI CONSERVAZIONE (RICORDATE CHE M È UN MATCHING, QUINDI 2 ARCHI DI M NON POSSONO CONDIVIDERE ALCUN ESTREMO).

POICHÉ $C = \sum_{(s,x) \in E'} c(e) = |X| \leq |V| = |V'| - 2$, $|E'| = |E| + |V|$ (G' CONTIENE UN ARCO DA s A OGNI VERTICE DI X E DA OGNI VERTICE DI Y A t, QUINDI UN ARCO PER OGNI VERTICE DI V) E CHE L'ALGORITMO DI FORD-FULKERSON IMPIEGA TEMPO $O(|G'| \cdot C)$ PER CALCOLARE IL FLUSSO MASSIMO DI G' (CHE CORRISPONDE ALLA MASSIMA TAGLIA DI UN MATCHING DI G), CONCLUDIAMO CHE IL PROBLEMA DEL CALCOLO DEL MATCHING DI TAGLIA MASSIMA RICHIEDE TEMPO $O(|G| \cdot m)$, CON $m = |V|$.

SEGMENTAZIONE DI IMMAGINI

LA SEGMENTAZIONE DI IMMAGINE È IL PROCESSO DI SUDDIVISIONE DI UNA IMMAGINE DIGITALE IN DIVERSE REGIONI COERENTI: SFONDO, OGGETTI IN PRIMO PIANO, ECC.

LA VERSIONE BASE DEL PROBLEMA RICHIEDE LA CLASSIFICAZIONE DEI PIXEL DI UNA IMMAGINE IN QUELLI APPARTENENTI ALLO SFONDO E QUELLI DEL PRIMO PIANO. CONSIDERIAMO L'IMMAGINE COME INSIEME DI PIXEL ORGANIZZATI IN UNA GRIGLIA. L'OBIETTIVO È ETICHETTARE OGNI PIXEL CON B, PER BACKGROUND (SFONDO), O CON F, PER FOREGROUND (PRIMO PIANO).

IN INGRESSO ABBIAMO I SEGUENTI DATI:

- PER OGNI PIXEL i UN VALORE b_i CHE STIMA LA NOSTRA CONFIDENZA CHE i FACCIA PARTE DELLO SFONDO;
- PER OGNI PIXEL i UN VALORE f_i CHE STIMA LA NOSTRA CONFIDENZA CHE i FACCIA PARTE DEL PRIMO PIANO;
- PER OGNI COPPIA DI PIXEL i e j ADIACENTI p_{ij} INDICA LA PENALIZZAZIONE IN CUI SI INCORRE SE SEPARIAMO I DUE PIXEL INSERENDOLI IN CLASSI DIFFERENTI.

VOGLIAMO OTTENERE UN PARTIZIONAMENTO DEI PIXEL NEI DUE INSIEMI B e F IN MODO DA MASSIMIZZARE LA QUANTITÀ

$$q(B,F) = \sum_{x \in B} b_x + \sum_{y \in F} f_y - \sum_{\substack{x \in B \\ y \in F \\ x \text{ adj } y}} p_{xy}$$

MASSIMIZZARE LA CONFIDENZA DELLA CLASSIFICAZIONE LIMITANDO AL MASSIMO LE DISCONTINUITÀ TRA SFONDO E PRIMO PIANO.

RIDUCIAMO IL PROBLEMA DI SEGMENTAZIONE OTTIMA AD UN PROBLEMA DI MINIMO TAGLIO DI UNA RETE DI FLUSSO.

OGNI IMMAGINE PUO' ESSERE FACILMENTE RAPPRESENTATA DA UN GRAFO NON ORIENTATO G I CUI VERTICI, CORRISPONDONO AI PIXEL E GLI ARCHI ALLE RELAZIONE DI ADIACENZA TRA I PIXEL NELLA GRIGIA.

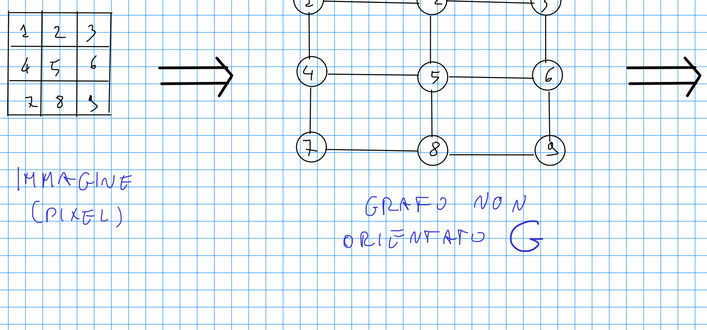
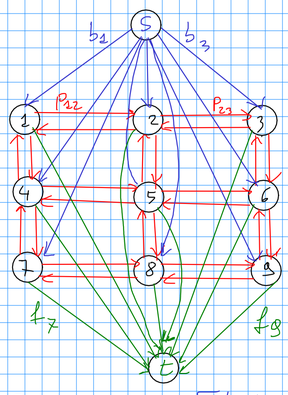


IMMAGINE (PIXEL) ⟹ GRAFO NON ORIENTATO G ⟹



RETE DI FLUSSO G'

DA G POSSIAMO DERIVARE UNA RETE DI FLUSSO G' AGGIUNGENDO:

- DUE NUOVI VERTICI s E t
- UN ARCO DA s AD OGNI PIXEL x DI CAPACITA' b_x
- UN ARCO DA OGNI PIXEL x A t DI CAPACITA' f_x
- A OGNI ALTRO ARCO (x,y) LA CAPACITA' P_{xy}.

PER RIDURRE IL PROBLEMA DI SEGMENTAZIONE AL TAGLIO MINIMO CI SERVE PRIMA TRASFORMARLO IN UN PROBLEMA DI MINIMIZZAZIONE.

SE IMPOSTIAMO

$$Q = \sum_{x \in V} b_x + \sum_{x \in V} f_x = \left(\sum_{x \in B} b_x + \sum_{y \in F} b_y \right) + \left(\sum_{x \in B} f_x + \sum_{y \in F} f_y \right)$$

E' IMMEDIATO OSSERVARE CHE

$$q(B,F) = Q - \sum_{y \in F} b_y - \sum_{x \in B} f_x - \sum_{\substack{x \in B \\ y \in F \\ x \text{ adj } y}} P_{xy}$$

$$= Q - \left(\sum_{y \in F} b_y + \sum_{x \in B} f_x + \sum_{\substack{x \in B \\ y \in F \\ (x,y) \notin E}} P_{xy} \right)$$

POICHE' Q E' COSTANTE (NON DIPENDE DA B E F), MASSIMIZZARE $q(B,F)$ E' EQUIVALENTE A MINIMIZZARE

$$q^*(B,F) = \sum_{y \in F} b_y + \sum_{x \in B} f_x + \sum_{\substack{x \in B \\ y \in F \\ x \text{ adj } y}} P_{xy}$$

LA PROPRIETA' SEGUENTE È IL FULCRO DELLA SOLUZIONE E METTE IN RELAZIONE I TAGLI IN G' CON LE SEGMENTAZIONI DELL'IMMAGINE

TEOREMA: SIA (A,D) UN QUALSIASI TAGLIO DI G'. ALLORA

$$c(A,D) = \varphi^*(A,D)$$

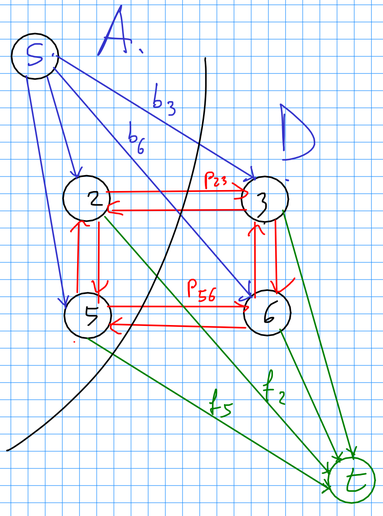
DIM.: Per definizione, la capacità di un taglio (A,D) è

$$c(A,D) = \sum_{\substack{(x,y)\in E' \\ x\in A \\ y\in B}} c(x,y)$$

con E' l'insieme degli archi della rete G'

Gli archi che attraversano il taglio (A,D) possono essere di 3 tipi:

1) Archi del tipo $(s,y)\in E'$ con $y\in D$
2) Archi del tipo $(x,t)\in E'$ con $x\in A$
3) Archi del tipo $(x,y)\in E'$ con $x\in A$ e $y\in D$



$$A = \{2,5\}$$

$$D = \{3,6\}$$

$$C(A,D) = \underbrace{(b_3+b_6)}_{\sum_{x\in D} b_x} + \underbrace{(f_2+f_5)}_{\sum_{x\in A} f_x} + \underbrace{(p_{23}+p_{56})}_{\sum_{\substack{(x,y)\in E' \\ x\in A \\ y\in D}} p_{xy}}$$

La capacità degli archi di Tipo 1) è pari a b_y; quella degli archi di Tipo 2) è f_x; quella degli archi di Tipo 3) è p_{xy}.
La somma della capacità di questi archi è quindi pari a:

$$\sum_{y\in D} b_y + \sum_{x\in A} f_x + \sum_{\substack{(x,y)\in E' \\ x\in A \\ y\in D}} p_{xy} = \varphi^*(A,D)$$

da cui segue la tesi.

DAL TEOREMA PRECEDENTE SEGUE CHE POSSIAMO CALCOLARE LA SEGMENTAZIONE OTTIMA DI UNA IMMAGINE COSTRUENDO PRIMA LA CORRISPONDENTE RETE DI FLUSSO G' E, SUCCESSIVAMENTE, CALCOLANDO IL TAGLIO MINIMO (B,F) DI G', UTILIZZANDO L'ALGORITMO DI FORD-FULKERSON. IL TAGLIO OTTENUTO, INFATTI, MINIMIZZA LA QUANTITÀ $\varphi^*(B,F)$ E, QUINDI, MASSIMIZZA $q(B,F)$.

PROBLEMI INTRATTABILI: LA CLASSE NP

Un problema X si dice **TRATTABILE** se esiste un algoritmo che risolve ogni sua istanza $x \in X$ in tempo **POLINOMIALE sulla dimensione** di x, cioè in tempo $O(n^k)$, dove $n = |x|$ e $k \geq 0$ è una costante positiva.

Un problema X che non è trattabile è detto **INTRATTABILE**.

La classe dei problemi trattabili, risolvibili in tempo polinomiale, è indicata con P.

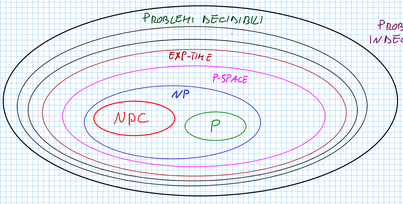
Un problema $X \in P$ se esiste un algoritmo che lo risolve in tempo polinomiale sulla sua dimensione.

Esistono molti problemi per cui è stata stabilita (dimostrata) la non-trattabilità, ma anche molti problemi per cui semplicemente non si è mai trovata una soluzione in tempo polinomiale

La classe di questi ultimi problemi è indicata con NPC (problemi NP-COMPLETI).

Trovare una soluzione efficiente (in tempo polinomiale) per un qualsiasi problema in NPC significa poter risolvere ogni problema in NPC in tempo polinomiale.

Esiste una gerarchia di classi di problemi, che permette di classificare i problemi decidibili in base alla loro difficoltà.



PROBLEMI INDECIDIBILI (formate da una macchina di Turing)

PROBLEMI DECIDIBILI:

- P = TEMPO POLINOMIALE DETERMINISTICO (percorso minimo in un grafo, ...)
- NP = TEMPO POLINOMIALE NON DETERMINISTICO (percorso massimo in un grafo, ...) (NPC)
- P-SPACE = SPAZIO POLINOMIALE (TSP, planning, ...)
- EXP-TIME = TEMPO ESPONENZIALE (scacchi, dama, ...)

Sono note le seguenti relazioni tra le classi

$$P \subseteq NP \subseteq \text{P-SPACE} \subseteq \text{EXP-TIME}$$

$$P \subset \text{EXP-TIME}$$

PROBLEMI DI DECISIONE E PROBLEMI DI OTTIMIZZAZIONE

La Teoria della complessità si concentra sullo studio della difficoltà dei problemi di decisione.

PROBLEMA DI DECISIONE: Problema la cui soluzione è SI/NO.

ESEMPIO

PATH$^E(G, v, u, k)$: Dato un grafo $G = \langle V, E \rangle$, due vertici v e u e un intero k, esiste un percorso da v a u in G con al più k archi?

Per ogni problema di ottimizzazione esiste un problema di decisione corrispondente.

ESEMPIO

MIN-PATH(G, v, u): Dato un grafo $G = \langle V, E \rangle$ e due vertici v e u calcolare la lunghezza del percorso minimo da v a u in G.

Il problema PATH$^E(G, v, u, k)$ è il problema di decisione corrispondente a MIN-PATH(G, v, u) nel seguente senso.
Se abbiamo un algoritmo per risolvere il problema MIN-PATH(G, v, u), allora possiamo ottenere un algoritmo per risolvere PATH$^E(G, v, u, k)$:

> Basta risolvere MIN-PATH(G, v, u). Se il risultato ottenuto è $\leq k$ rispondiamo SI, altrimenti rispondiamo NO.

Intuitivamente, PATH$^E(G, v, u, k)$ non è più difficile da risolvere di MIN-PATH(G, v, u).
Studiare la difficoltà di PATH$^E(G, v, u, k)$ ci dà informazione anche sulla difficoltà di MIN-PATH(G, v, u)

In questo caso vale anche il contrario. Se sappiamo risolvere PATH$^E(G, v, u, k)$ per ogni k, allora possiamo definire il seguente algoritmo per MIN-PATH(G, v, u):

> Risolvere PATH$^E(G, v, u, k)$ per valori di k crescenti da 0 a $|V|-1$. Il risultato desiderato è il più piccolo k per cui otteniamo risposta SI per il problema di decisione.
> (sono sufficienti $\log(|V|)$ domande a PATH$^E(G, v, u, k)$)

Se consideriamo, invece, il problema di ottimizzazione:

MAX-PATH(G, v, u): Dato un grafo $G = \langle V, E \rangle$ e due vertici v e u calcolare la lunghezza del massimo percorso semplice da v a u in G.

Il corrispondente problema di decisione è:

PATH$^E(G, v, u, k)$: Dato un grafo $G = \langle V, E \rangle$, due vertici v e u e un intero k, esiste un percorso da v a u in G con almeno k archi?

MISURA DELLA DIMENSIONE DI ISTANZE DI PROBLEMI

La complessità di un algoritmo in tempo o in spazio è misurata come funzione della dimensione dell'input, ovvero l'input è una qualche codifica dell'istanza corrente del problema che si vuole risolvere.
La codifica per un problema X è una funzione c che associa ad ogni istanza $x \in X$ una rappresentazione concreta $c(x)$ di x che può essere elaborata da un calcolatore.

ESEMPI:
- Una istanza del problema del calcolo del K-esimo numero di Fibonacci è costituita dal numero K la cui rappresentazione standard in informatica è una stringa binaria di lunghezza $\log K$.
- Una istanza del problema $\text{PATH}^3(G, n, u, k)$ è costituita da un grafo $G = \langle V, E \rangle$, la cui rappresentazione può essere una matrice di adiacenza o un insieme di liste di adiacenza, entrambe di dimensione non superiore a $|V|^2$. In ultima analisi, però, entrambe queste 2 rappresentazioni vengono codificate come stringhe di bit di lunghezza polinomiale su $|V|$ (esattamente $\leq (|V|^2)$).

> La complessità di un algoritmo o di un problema può dipendere dal tipo di codifica scelta.

ESEMPIO: Sia dato un algoritmo per il calcolo dell'n-esimo numero di Fibonacci impiega Tempo lineare sul valore n.
Consideriamo le seguenti codifiche:
a) n rappresentato in forma di stringa unaria
$n = 100$ $\quad c_1(n) = \underbrace{1111\ldots1}_{n}$
b) n rappresentato in forma di stringa binaria
$n = 100$ $\quad c_2(n) = 01100100$
c) n rappresentato in forma di stringa decimale
$n = 100$ $\quad c_3(n) = 100$

In base alla rappresentazione dell'input quanto tempo impiega l'algoritmo in funzione della dimensione della codifica dell'input?

$k = |c_1(n)| = n \implies T(k) = \Theta(k)$
$k = |c_2(n)| = \log_2 n \implies n = 2^k$ e $T(k) = \Theta(c_2^k)$
$k = |c_3(n)| = \log_{10} n \implies n = 10^k$ e $T(k) = \Theta(10^k)$

Con entrambe le codifiche c_2 e c_3, l'algoritmo impiega tempo esponenziale.

> Assumeremo, d'ora in poi, sempre una codifica dell'input come stringa binaria.

ALGORITMI DI SOLUZIONE VERSUS ALGORITMI DI VERIFICA

Dato un problema di decisione X, un algoritmo di soluzione per X decide, per ogni istanza $x \in X$, se x è una istanza SI o NO per il problema.
Un algoritmo di verifica per X, invece, verifica se un qualche "certificato" è fornito in ingresso insieme all'istanza $x \in X$, dimostra che l'istanza $x \in X$ è un'istanza SI del problema X.

Per esempio, dato il problema
$\text{PATH}^4(G, n, u, k)$: Dato un grafo $G = \langle V, E \rangle$, due vertici u e v in G e un intero k, esiste un percorso da u a v in G con al più k archi?

Un algoritmo di verifica per PATH^4 prenderebbe in ingresso $\langle G, n, u, k \rangle$ e una sequenza π di vertici e verificherebbe se le seguenti condizioni:
a) π è un percorso in G
b) π va da u a v
c) π contiene non più di k archi

Se a), b) e c) sono Tutte e 3 vere, allora π è un certificato corretto per $\text{PATH}^4(G, n, u, k)$ e quindi l'algoritmo risponde che $\langle G, n, u, k \rangle$ è una istanza SI di PATH^4.
Altrimenti π non è un certificato corretto. In questo caso, non può dire se $\langle G, n, u, k \rangle$ è una istanza SI o una istanza NO di PATH^4.

In genere, decidere se un certificato è corretto o meno è più facile che decidere se $x \in X$ è una istanza SI o NO.

ESEMPIO
- Dato un grafo G, un ciclo Hamiltoniano è un ciclo semplice (ovvero ripetono Tranne primo vertice che è uguale all'ultimo) che visita Tutti i vertici di G.
- Un grafo G si dice grafo Hamiltoniano se contiene un ciclo Hamiltoniano.

> HAM-CYCLE(G): G è un grafo Hamiltoniano?

HAM-CYCLE ammette un algoritmo di verifica polinomiale.
Un certificato c per HAM-CYCLE(G) è una sequenza di vertici $c = v_1, v_2, \ldots, v_m$ (con $m = |V|$ e $G = \langle V, E \rangle$). È facile definire un algoritmo che in tempo polinomiale su $|G|$ e $|G|$ verifica se:
a) c è un percorso in G
b) $v_1 = v_m$
c) i primi m vertici in c sono Tutti Tra loro diversi

Se tutte e 3 le verifiche hanno successo, allora c è un ciclo Hamiltoniano in G e, quindi, G è una istanza SI del problema HAM-CYCLE.

LE CLASSI P E NP

> La classe P è la classe dei problemi per cui esiste un algoritmo di soluzione, che impiega Tempo polinomiale $O(n^m)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

> La classe NP è la classe dei problemi per cui esiste un algoritmo di verifica, che impiega Tempo polinomiale $O(n^m)$, con $k \geq 0$ e n pari alla dimensione dell'istanza in esame.

Formalmente, $X \in NP$ se esiste un algoritmo $B_X(c, y)$ da eseguire in ingresso 2 parametri, $x \in X$ e c, tale che:
- B_X impiega Tempo polinomiale
- per ogni $x \in X$, x è una istanza SI per X se e soltanto se esiste un certificato c, con $|c| \leq n^k$, $k \geq 0$ e $n = |x|$, tale che $B_X(x, c) = \text{SI}$.

P è la classe di problemi risolvibili in Tempo polinomiale, NP è la classe dei problemi verificabili in Tempo polinomiale.
L'esistenza del verificatore $B_X(c, y)$ permette di risolvere il problema X generando, dato $x \in X$, tutti i possibili certificati c, con $|c| \leq |x|^k$, ed eseguendo $B_X(x, c)$ per ciascun certificato. Ne risulta un algoritmo di "forza bruta" che impiega Tempo esponenziale.

> TEOREMA: $P \subseteq NP$

Dim
Se $X \in P$ Allora esiste un algoritmo A_X che risolve ogni istanza $x \in X$ in Tempo polinomiale rispetto a $|x|$.
Possiamo definire un verificatore $B_X(c, y)$ che, dati in ingresso x e c, ignora c ed esegue A_X su input x.
Se $A_X(x)$ risponde SI, $B_X(x, c)$ risponde SI, altrimenti $B_X(x, c)$ risponde NO. $B_X(x, c)$ risponde SI se e soltanto se x è un'istanza SI per X e impiega Tempo polinomiale, poiché A_X impiega Tempo polinomiale. □

DIFFICOLTÀ RELATIVA E RIDUZIONI

Quando possiamo dire che un problema X è più difficile di un problema Y?

Dati 2 problemi X e Y scriviamo

$$Y \leq_p X$$

per indicare che X è difficile almeno quanto Y.

$Y \leq_p X$ si legge

"Y è riducibile a X in tempo polinomiale"

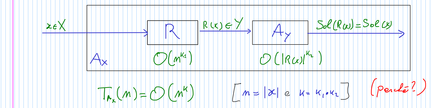
Dove:

Y è riducibile a X se esiste un algoritmo R, che trasforma una qualsiasi istanza y del problema Y in una istanza $R(y)$ del problema X, tale che le soluzioni di $R(y)$ e di y coincidano.

Se R impiega tempo polinomiale sulla dimensione di y per trasformare y in $R(y)$ allora R è una **riduzione polinomiale**.

TEOREMA Se $X \leq_p Y$ e $Y \in P$, allora $X \in P$.

Dim Per definizione, se $X \leq_p Y$, allora esiste un algoritmo R che in tempo polinomiale trasforma una qualsiasi istanza $x \in X$ in una istanza $y = R(x) \in Y$, tale che $Sol(x) = Sol(y)$. Se $Y \in P$, l'istanza $R(x)$ è risolvibile in tempo polinomiale da un algoritmo A_Y. Quindi:



$T_{A_X}(m) = O(m^k)$ $[m = |x| \text{ e } k, k' \text{ costanti}]$ (perché?)

A_X è un algoritmo che risolve in tempo polinomiale ogni istanza $x \in X$ eseguendo prima R e poi l'algoritmo A_Y che risolve Y.

Quindi $X \in P$ □

Il contrapposto del Teorema ci dice che:

Se $X \leq_p Y$ e $X \notin P$, allora $Y \notin P$.

Se possiamo ridurre in tempo polinomiale un problema X **non risolvibile in tempo polinomiale** a un dato problema Y, allora anche Y **non è risolvibile in tempo polinomiale**.

ESEMPIO:

Dato un grafo $G = \langle V, E \rangle$, un **vertex cover** di G è un insieme di vertici $V' \subseteq V$ t.c. per ogni $(v,u) \in E$, $v \in V'$ oppure $u \in V'$. (V' copre ogni arco di G).

VERTEX-COVER (G, k): G contiene un vertex cover di dimensione al più k ($\leq k$)

Dato un grafo $G = \langle V, E \rangle$, un **independent set** di G è un insieme di $V' \subseteq V$ t.c. per ogni $v, u \in V'$, $(v,u) \notin E$. (V' non contiene vertici connessi da archi di G)

INDEPENDENT-SET (G, k): G tiene un independent set di dimensione almeno k ($\geq k$)

TEOREMA. Valgono entrambe le seguenti affermazioni

VERTEX-COVER $\leq_p$ INDEPENDENT-SET

e

INDEPENDENT-SET $\leq_p$ VERTEX-COVER

Dim.

Consideriamo un grafo arbitrario $G = \langle V, E \rangle$

È facile osservare che un sottoinsieme di vertici $V' \subseteq V$ è un **independent set** se e solo se $V \setminus V'$ è un **vertex cover**.

Se V' è un **independent set** e $(v,u) \in E$, allora almeno uno tra v e u **non appartiene a** V'. Quindi almeno uno tra v e u **appartiene a** $V \setminus V'$. Segue che ogni arco in E ha almeno uno dei suoi due estremi in $V \setminus V'$, che è quindi un **vertex cover** di G.

Viceversa, se $V \setminus V'$ è un **vertex cover**, allora ogni arco $(v,u) \in E$ ha almeno uno dei 2 estremi in $V \setminus V'$. Ma quindi nessuna coppia di vertici in V' può essere connessa da un arco in E. Segue che V' è un **independent set** di G.

Quindi, se abbiamo un algoritmo A_{IS} per risolvere INDEPENDENT-SET possiamo risolvere VERTEX-COVER.

Data una istanza $\langle G, k \rangle$ per VERTEX-COVER è sufficiente costruire l'istanza $\langle G, |V| - k \rangle$ per INDEPENDENT-SET e risolverla tramite A_{IS}. Se A_{IS} restituisce SI allora $\langle G, k \rangle$ è una istanza SI di VERTEX-COVER.

Poiché $\langle G, |V| - k \rangle$ può essere banalmente costruita in tempo polinomiale a partire da $\langle G, k \rangle$, otteniamo

VERTEX-COVER $\leq_p$ INDEPENDENT-SET

In maniera analoga (duale) si dimostra che

INDEPENDENT-SET $\leq_p$ VERTEX-COVER □

TEOREMA Se $X \leq_p Y$ e $Y \leq_p Z$, allora $X \leq_p Z$

Dim. Siano dati un algoritmo R_{XY}, che trasforma una qualsiasi istanza $x \in X$ in una istanza $R_{XY}(x) \in Y$, e un algoritmo R_{YZ}, che trasforma una qualsiasi istanza $y \in Y$ in una istanza $R_{YZ}(y) \in Z$. Allora possiamo definire un algoritmo R_{XZ} che, dato in ingresso una qualsiasi istanza $\overline{x} \in X$, prima applica l'algoritmo R_{XY} per ottenere $\overline{y} = R_{XY}(\overline{x})$ in tempo polinomiale se $|\overline{x}|$ e poi esegue l'algoritmo R_{YZ} su $\overline{y}$, ottenendo $\overline{z} = R_{YZ}(\overline{y})$ in tempo polinomiale su $|\overline{y}|$. Poiché $|\overline{y}|$ è polinomiale su $|\overline{x}|$ (perché?), su $|\overline{y}|$ $\overline{z}$ è ottenuto in tempo polinomiale su $|\overline{x}|$. □

PROBLEMI NP-COMPLETI

Un problema X si dice **NP-COMPLETO** se:

1) $X \in NP$
2) $\forall\, Y \in NP,\ Y \leq_p X$ (**NP-DIFFICILE**)

La classe dei problemi NP-COMPLETI è indicata con **NPC**.

Intuitivamente, i problemi NP-COMPLETI rappresentano i problemi più difficili della classe NP.

Se trovassi un algoritmo in grado di risolvere un qualsiasi problema in NPC, allora tutti i problemi in NP sarebbero risolvibili in tempo polinomiale (da chiarire), e quindi varrebbe $P = NP$.

A tutt'oggi uno è solo alcuno algoritmo in grado di risolvere un qualche problema in NPC in tempo polinomiale.

SODDISFACIBILITÀ DEI CIRCUITI

PORTE LOGICHE

Ogni filo trasporta un valore binario (0/1).

x	NOT
0	1
1	0

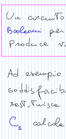
x	y	AND
0	0	0
0	1	0
1	0	0
1	1	1

x	y	OR
0	0	0
0	1	1
1	0	1
1	1	1

Ogni porta logica calcola una funzione Booleana specifica.

Un CIRCUITO COMBINATORIO è un insieme di porte logiche tra loro connesse dai fili.

C_α



C_α calcola la funzione Booleana $f_{C_\alpha}(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$

$Y = f_{C_\alpha}(x_1, x_2)$

Un circuito C si dice **SODDISFACIBILE** se esistono valori Booleani per i fili di input tali che il filo di output produce valore 1.

Ad esempio il circuito C_α nella figura sopra è soddisfacibile: infatti, in ingresso $x_1 = 1 \in x_2 = 0$ ($\in x_3 = \dots = x_{101} = 1$) restituisce $y = 1$.

C_α calcola la funzione Booleana $f_{C_\alpha}(x_1) \Rightarrow$

x_1	f_{C_α}
0	0
1	0

Un circuito C è soddisfacibile se la funzione Booleana che calcola è diversa dalla funzione costante 0.

CIRCUIT-SAT: Decidere se il circuito C in ingresso è soddisfacibile

TEOREMA CIRCUIT-SAT $\in$ NP

Dim. Dato un circuito C, un certificato è un assegnamento x di valori 0/1 a ogni filo di input del circuito. Chiaramente $|x| \leq |C|$, quindi la dimensione del certificato è polinomiale su $|C|$. Verificare che x è un certificato corretto può essere facilmente effettuato in tempo polinomiale su $|C|$, valutando un porta logica di C. Ognuna di tali verifiche impiega tempo per al più n^2 fili in ingresso alla porta, tale $n^2 \in |C|$. Il n^2 di porte di C è chiaramente $\leq |C|$. Quindi, il tempo richiesto è $\leq |C|^2$, polinomiale in $|C|$. □

TEOREMA CIRCUIT-SAT $\in$ NPC

Dim. 1) CIRCUIT-SAT $\in$ NP

Il Teorema precedente ci dà l'appartenenza a NP. Per ottenere il risultato senza fare sedere da ogni problema $X \in NP$ può essere molto a CIRCUIT-SAT.

2) CIRCUIT-SAT è NP-DIFFICILE

Si deve dimostrare che dato $x \in X$ esiste un circuito C_x, costruito in tempo polinomiale su $|x|$, tale che:

x è un'istanza SI sse C_x è soddisfacibile (1)

Dato $X \in NP$, esiste un algoritmo polinomiale A_X t.c. per ogni istanza $x \in X$, vale che

x è un'istanza SI sse $\exists c.\ A_X(x, c) = 1$ (2)

Chiaramente $|c|$ è polinomiale in $|x|$ in quanto in tempo polinomiale $t(n)$ si possono leggere (o inviare o scrivere) al massimo un numero di simboli polinomiali. Quindi $|c| \leq t(n)$. Assumiamo una codifica binaria degli input x, in particolare di c.

La dimostrazione costruisce un circuito C_x con input lunghi d bit di lunghezza $|c|$, tale che

$C_x(c) = 1$ sse $A_X(x, c) = 1$ (3)

È facile osservare che (1)+(2)+(3) implicano la tesi. Non è difficile osservare che, una volta fissati X e x, $A_X(x, c)$ calcola una funzione Booleana di $k = |c|$ variabili Booleane.

Quindi, una volta fissata l'istanza x, il comportamento di $A_X(x, c)$ può essere simulato da un circuito.

[Una funzione Booleana in k variabili è calcolata da un circuito con k fili di input.]

Si può dimostrare che la dimensione di C_x può essere nuovamente polinomiale su $|c| \leq t(n)$, quindi, ovviamente polinomiale su $|x|$.

Assumo allora ottenuto un circuito C_x che produce 1 (è soddisfacibile) se e solamente se esiste una stringa di bit c t.c. $A_X(x, c) = 1$, il che dimostra (3). □

SODDISFACIBILITÀ BOOLEANA

Siano date n variabili Booleane $X = \{x_1, x_2, \dots, x_n\}$.

Un **Termine** Booleano su X è una delle variabili x_i o la sua negazione $\bar{x}_i$ ($i \in_p \forall x_i$) [Letterali sono anche detti **Letterali**].

Una **clausola** è semplicemente una disgiunzione di Termini su X

$$t_i \vee t_i \vee \dots \vee t_k$$

con $t_i \in \{x_1, \dots, x_n, \bar{x}_1, \dots, \bar{x}_n\}$

Una **formula** Booleana in forma CNF (CONJUNCTIVE NORMAL FORM) è una congiunzione di clausole

$$C_\alpha \wedge C_\beta \wedge \dots \wedge C_\kappa$$

Una formula CNF è soddisfacibile se esiste un assegnamento di valori 0 o vero alle variabili in X tale che la formula valuta a 1.

SAT: Data una formula CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

Una **formula** Booleana in forma 3-CNF è una congiunzione di clausole

$$C_\alpha \wedge C_\beta \wedge \dots \wedge C_\kappa$$

Tali che ogni clausola C_i contiene esattamente 3 Termini

$$C_i = (\ell_i \vee \ell_i' \vee \ell_i'')$$

3-SAT: Data una formula 3-CNF ϕ in ingresso, decidere se ϕ è soddisfacibile.

TEOREMA, 3-SAT $\in$ NPC

Dim. 1) 3-SAT $\in$ NP

Data una formula 3-CNF ϕ, un certificato è un assegnamento σ: $X \to \{0,1\}$, che associa un valore Booleano a ogni variabile di ϕ.

Un algoritmo di verifica $A(\phi, \sigma)$ semplicemente sostituisce in ϕ ad ogni variabile x_i il valore $\sigma(x_i)$

Esempio:

$\phi = (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee x_4) \wedge (\bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4) \wedge (x_1 \vee x_3 \vee x_4)$

$\sigma = (x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1)$

$\Downarrow$

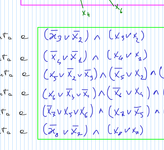
$\phi_\sigma = (1 \vee 0 \vee 0) \wedge (0 \vee 1 \vee 0) \wedge (0 \vee 0 \vee 1) \wedge (1 \vee 0 \vee 1)$

Ovviamente, l'algoritmo $A(\phi, \sigma)$ verifica che in ciascuna clausola di ϕ_σ sia presente almeno un 1.

2) CIRCUIT-SAT $\leq_p$ 3-SAT

Dobbiamo come procedere che, dato in ingresso un circuito combinatorio C, costruisce una formula 3-CNF ϕ_c. L'idea è di introdurre una variabile booleana per ogni filo di input, output e per ogni porta logica del circuito.

ESEMPIO:



$x_3 = \bar{x}_2$ equivalente a	$(\bar{x}_2 \vee \bar{x}_3) \wedge (x_2 \vee x_3)$	ϕ_1
$x_4 = \bar{x}_1$ equivalente a	$(\bar{x}_1 \vee \bar{x}_4) \wedge (x_1 \vee x_4)$	ϕ_2
$x_5 = x_1 \wedge x_3$ equivalente a	$(x_1 \vee \bar{x}_5 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_5) \wedge (\bar{x}_3 \vee x_5)$	ϕ_3
$x_6 = x_1 \wedge x_4$ equivalente a	$(\bar{x}_1 \vee \bar{x}_4 \vee \bar{x}_6) \wedge (x_1 \vee x_6) \wedge (x_4 \vee x_6)$	ϕ_4
$x_7 = x_5 \vee x_6$ equivalente a	$(\bar{x}_5 \vee x_6 \vee x_7) \wedge (x_5 \vee \bar{x}_7) \wedge (x_6 \vee \bar{x}_7)$	ϕ_5
$x_8 = x_2$ equivalente a	$(\bar{x}_8 \vee x_8) \wedge (x_8 \vee \bar{x}_8)$	ϕ_6

$$\phi_C = \phi_6 \wedge \phi_3 \wedge \phi_4 \wedge \phi_5 \wedge \phi_1 \wedge \phi_2$$

In generale,

$$\Longrightarrow \quad (\bar{x}_i \vee x_k) \wedge (x_j \vee \bar{x}_k)$$

$$\Longrightarrow \quad (x_j \vee \bar{x}_i \vee \bar{x}_k) \wedge (\bar{x}_j \vee x_i) \wedge (\bar{x}_j \vee x_k)$$

$$\Longrightarrow \quad (\bar{x}_j \vee x_i \vee x_k) \wedge (x_j \vee \bar{x}_i) \wedge (x_j \vee \bar{x}_k)$$

ϕ_C è, dunque, la congiunzione della formule ottenute. Poiché il n° di variabili aggiunte è pari al n° di porte logiche di C, che è $\leq |C|$, e ogni formula ϕ_j ha dimensione costante (indipendente da $|C|$), la dimensione della formula ϕ_C è $O(|C|)$. La formule ϕ_C può essere banalmente costruita in tempo polinomiale su $|C|$. Dobbiamo dimostrare che:

ϕ_C è soddisfacibile $\Longleftrightarrow$ C è soddisfacibile

$\Longrightarrow$ Se ϕ_C è soddisfacibile, esiste un assegnamento per le variabili booleane che rende vera ogni formula ϕ_j (perché ϕ_C è la congiunzione delle ϕ_j).
Ogni formula ϕ_j assegna che il valore della variabile aggiuntiva x_j è identico al valore di output della corrispondente porta logica. Quindi il circuito C produce in output lo stesso valore di verità della variabile booleana che rappresenta il filo di output di C. Tale variabile deve essere impostata a 1 affinché ϕ_C sia soddisfatta, per cui è stata costruita ϕ_C

$\Longleftarrow$ Se C è soddisfacibile, l'assegnamento ai fili di input può essere estaso alle variabili booleane di ϕ_C, valutando il valore prodotto da ciascuna porta logica di C e assegnando lo stesso valore alla corrispondente variabile booleana in ϕ_C. L'assegnamento così ottenuto soddisfa ϕ_C.

ϕ_C non è in forma 3-CNF (una è in forma CNF con al massimo 3 letterali per clausola).
Può facilmente essere messo in formato 3-CNF aggiungendo 2 variabili booleane, p e q, in più:

$$(x_i \vee x_j) \Longleftrightarrow (x_i \vee x_j \vee p) \wedge (x_i \vee x_j \vee \bar{p})$$

$$(x_i) \Longleftrightarrow (x_i \vee p \vee q) \wedge (x_i \vee p \vee \bar{q}) \wedge (x_i \vee \bar{p} \vee q) \wedge (x_i \vee \bar{p} \vee \bar{q})$$

Effettuando questa sostituzione in ϕ_C, otteniamo una formula 3-CNF equivalente di dimensione polinomiale su $|C|$. Da ciò discende la tesi. □

Dato un grafo $G = (V, E)$, un **independent set** di G è un insieme di $V' \subseteq V$ t.c., per ogni $u, v \in V'$, $(u, v) \notin E$.

INDEPENDENT-SET (G, k) G contiene un independent set L dimensione al più k ($\in \mathbb{N}$)

TEOREMA. INDEPENDENT-SET è NP-COMPLETO

Dim.

1) INDEPENDENT-SET $\in NP$. Un certificato è un sottoinsieme $c \subseteq V$ di vertici. Verificare che $|c| \leq k$ e che ogni arco $(u, v) \in E$ ha almeno un estremo fuori da c può essere fatto in tempo $O(|E| \cdot |c|) = O(|E| \cdot |V|)$, quindi in tempo polinomiale sulla dimensione del grafo G ($|E| \leq |V| \times |V|$).

2) 3-SAT $\leq_p$ INDEPENDENT-SET

Dato la formula 3-CNF ϕ, costruiamo un grafo G_ϕ in cui:
- un gruppo di 3 vertici per ogni clausola C_i in ϕ da cui sono i vertici $V_i = \{v_i^1, v_i^2, v_i^3\}$

$$V = \bigcup_{C_i} V_i$$

- ciascuno dei 3 letterali della clausola C_i etichetta uno dei vertici di V_i

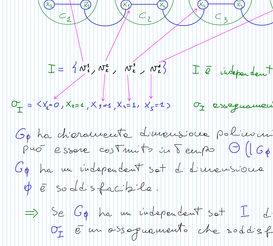
$$C_i = (l_i^1 \vee l_i^2 \vee l_i^3) \quad \Longrightarrow \quad lab_i(v_i^1) = l_i^1,\ lab_i(v_i^2) = l_i^2,\ lab_i(v_i^3) = l_i^3$$

- un arco tra due vertici $v \in V$ e $u \in V$ se
  - v e u appartengono allo stesso gruppo V_i
  - v e u sono etichettati con letterali opposti (x_i e $\bar{x}_i$).

$$E = \{(u, v) \mid (u, v) \in V^2 \text{ in cui } \exists\, i, j \text{ t.c. } (i \neq j) \wedge (lab_i(u) = \overline{lab_j(v)})\}$$

ESEMPIO:

$$\phi = \underbrace{(\bar{x}_1 \vee x_2 \vee x_3)}_{C_2} \wedge \underbrace{(x_1 \vee \bar{x}_2 \vee \bar{x}_3)}_{C_1} \wedge \underbrace{(x_1 \vee \bar{x}_2 \vee x_3)}_{C_3} \wedge \underbrace{(x_2 \vee x_3 \vee \bar{x}_1)}_{C_4}$$



$I = \{x_1^2, x_2^1, \bar{x}_2^2, \bar{x}_3^3\}$ I è independent set di dimensione 4

$\sigma_I = (x_1 = 0, x_2 = 1, x_3 = 1)$ σ_I assegnamento che soddisfa ϕ

G_ϕ ha chiaramente dimensione polinomiale su $|\phi|$ e può essere costruito in tempo $\Theta(|G_\phi|)$
G_ϕ ha un independent set I di dimensione k se e solo se ϕ è soddisfacibile.

$\Longrightarrow$ Se G_ϕ ha un independent set I di dimensione k e σ_I è un assegnamento che soddisfa ϕ. Infatti I contiene un vertice per ogni clausola e una può contenere vertici etichettati con letterali opposti (x_i e $\bar{x}_i$). σ_I rende vero i letterali dei vertici in I, quindi soddisfa ciascuna clausola e quindi ϕ.

$\Longleftarrow$ Se ϕ è soddisfacibile esiste un assegnamento σ che rende vero almeno un letterale per ogni clausola. Costruiamo I a partire da σ selezionando un vertice in ciascun V^i etichettato con un letterale reso vero da σ. I ha dimensione k ed è un independent set G_ϕ. Infatti, se così non fosse, ci dovrebbe essere un arco tra due vertici contenuti in I, e basso essi v e u. Per come è stato costruito I, $v \in V^i$ e $u \in V^j$ con $i \neq j$ (v e u non possono appartenere allo stesso gruppo di vertici). Ma allora, se $(v, u) \in E$, v e u devono avere come etichette due letterali opposti. Tali letterali non possono essere resi entrambi veri da σ. Quindi v e u non possono essere entrambi in I. □

COROLLARIO. VERTEX-COVER è NP-COMPLETO

Dim. Poiché INDEPENDENT-SET $\leq_p$ VERTEX-COVER e poiché INDEPENDENT-SET $\in$ NPC.

SUBSET-SUM (V, W): Dato un insieme di numeri naturali $V = \{a_1, \dots, a_n\}$ esiste un sottoinsieme di V la cui somma è pari a W?

TEOREMA. SUBSET-SUM è NP-COMPLETO

Dim.

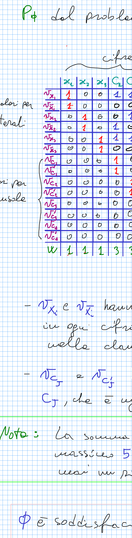
1) SUBSET-SUM $\in$ NP: Un certificato c è un sottoinsieme di numeri in V. Possiamo verificare che $c \in V$ e che la somma dei numeri in c è $= W$ in tempo polinomiale su $|V|$ e $\log W$.

2) 3-SAT $\leq_p$ SUBSET-SUM

Data una formula 3-CNF ϕ con variabili booleane $\{x_1, \dots x_n\}$ e k clausole, $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$, costruiamo in tempo polinomiale un'istanza di SUBSET-SUM come segue:

$$\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (\overline{x_1} \vee \overline{x_3} \vee x_4) \wedge (x_1 \vee x_3 \vee \overline{x_4})$$

Costruiamo in tempo polinomiale su $|\phi|$ una istanza P_ϕ del problema SUBSET-SUM come segue:



- 2 numeri per ogni variabile
  - v_{x_i}: per il letterale x_i
  - $v_{\overline{x_i}}$: per il letterale $\overline{x_i}$
- 2 numeri per ogni clausola
  - v_{c_j}: per la clausola C_j
  - v'_{c_j}: per la clausola C_j
- Ogni numero ha:
  - 1 cifra decimale per ogni variabile
  - 1 cifra decimale per ogni clausola

- v_{x_i} e $v_{\overline{x_i}}$ hanno un 1 nella cifra x_i (colonna x_i) e in ogni cifra C_j t.c. x_i (rispettivamente $\overline{x_i}$) occorre nella clausola C_j. Le rimanenti cifre sono 0
- v_{c_j} e v'_{c_j} hanno 0 ovunque, tranne nella cifra C_j, che è uguale a 1

Nota: La somma delle cifre in ogni colonna è al massimo 5. Nessuna somma di valori richiede mai un riporto.

ϕ è soddisfacibile $\iff$ P_ϕ è un'istanza sì del problema SUBSET-SUM

$\Rightarrow$ Se ϕ è soddisfacibile, sia σ un assegnamento che rende ϕ vera. Costruiamo un certificato per P_ϕ come segue. c contiene v_{x_i} se $\sigma(x_i) = 1$ e $v_{\overline{x_i}}$ se $\sigma(x_i) = 0$. Quindi, per ogni variabile booleana x_i:

- Uno solo tra v_{x_i} e $v_{\overline{x_i}}$ viene selezionato in c

Poiché σ soddisfa ϕ, rende vero almeno un letterale per ogni clausola C_j. Per ciascun $1 \leq j \leq k$,

- Se ϕ rende vero solo un letterale in C_j, vengono aggiunti a c sia v_{c_j} che v'_{c_j}.
- Se ϕ rende veri 2 letterali in C_j, viene aggiunto a c solo v_{c_j}
- Se ϕ rende veri 3 letterali di C_j, né v_{c_j} né v'_{c_j} fanno parte di c.

È facile osservare che la somma dei valori inseriti in c risulta in un numero che ha tutte le cifre corrispondenti alle variabili uguali a 1 e tutte le cifre corrispondenti alle clausole uguali a 3. Quindi c è un certificato corretto per P_ϕ.

$\Leftarrow$ Dato un certificato corretto c per P_ϕ, allora uno solo tra v_{x_i} e $v_{\overline{x_i}}$ può essere presente in c. Quindi possiamo definire σ come

$$\sigma(x_i) = \begin{cases} 1 & \text{se } v_{x_i} \in c \\ 0 & \text{se } v_{\overline{x_i}} \in c \end{cases}$$

Poiché la somma dei valori in c deve essere uguale a W, ogni cifra corrispondente a una clausola in quella somma deve essere uguale a 3. Quindi, per ciascuna clausola C_j, almeno uno dei valori v_ℓ (con ℓ un letterale di ϕ) presente in c deve avere un 1 nella sua cifra C_j. Questo significa che quel letterale è reso vero da σ e rende soddisfatta la clausola C_j. Essendo ogni clausola soddisfatta da σ, ϕ è soddisfacibile. □

Il problema dello zaino 0/1 ha un analogo problema decisionale:

KNAPSACK: Dato in ingresso un insieme $I = \{1, 2, \dots, n\}$ di oggetti, ciascuno con peso w_i e valore v_i, un numero naturale W e un numero naturale K, esiste un sottoinsieme di oggetti di I con peso complessivo $\leq W$ e valore complessivo $\geq K$.

TEOREMA. KNAPSACK è NP-COMPLETO

Dim.

1) KNAPSACK $\in$ NP è lasciato come esercizio

2) SUBSET-SUM $\leq_p$ KNAPSACK

Data un'istanza di SUBSET-SUM (V, W) con $X = \{x_1, \dots, x_n\}$, definiamo una istanza di KNAPSACK come segue $(O, \{\beta\}, \{\omega\}, W, K)$ dove

$O = \{1, 2, \dots, n\}$	è l'insieme di oggetti
$p_i = x_i$	per ogni $1 \leq i \leq n$
$v_i = x_i$	per ogni $1 \leq i \leq n$
$k = W$	(valori e pesi coincidono)

È immediato verificare che:

esiste uno zaino di capacità massima W il cui valore è $K = W$

se e soltanto se

esiste un sottoinsieme di X la cui somma è W.

□

ALGORITMI DI APPROSSIMAZIONE

Per i problemi **non trattabili** è poco probabile (o impossibile) poter definire algoritmi efficienti, in grado di risolvere in maniera esatta istanze di dimensione medio-grande.

In questi casi ci si può spesso accontentare di ottenere **soluzioni sufficientemente vicine** a quella esatta.

Se il problema di interesse è un problema di ottimizzazione, si può provare a definire un algoritmo che produce una **soluzione sub-ottima**, ma "abbastanza buona", ma impiegando **tempo polinomiale**.

Come valutiamo se una soluzione è "abbastanza buona"?

Supponiamo che X sia un problema di ottimizzazione in cui vogliamo ottenere una soluzione che **minimizza** il costo e sia C^* il costo della soluzione ottima.

Sia, invece, C il costo di una sub-ottima. Chiaramente $C^* \leq C$.

Il rapporto $\frac{C}{C^*}$, detto **rapporto di approssimazione**, misura quanto lontano dall'ottimo è la soluzione sub-ottima (approssimata).

Tale rapporto è sempre ≥ 1 e se $\frac{C}{C^*} = 1$, allora le due soluzioni sono entrambe ottime.

In generale, il rapporto di approssimazione può dipendere dalla dimensione $n = |x|$ dell'istanza $x \in X$ che vogliamo risolvere.

Dato un'istanza $x \in X$ e una funzione $\rho(n)$, diciamo che un algoritmo è una **$\rho(n)$-approssimazione** se

$$\frac{C}{C^*} \leq \rho(n) \quad \text{con } n = |x|$$

Analogamente, se X è un problema di massimizzazione, il rapporto di approssimazione è indicato con $\frac{C^*}{C}$, che è sempre ≥ 1.

In generale, quindi, dato un problema di ottimizzazione X e una funzione $\rho(n)$, un algoritmo si dice

$\rho(n)$-approssimato se

$$\max\left(\frac{C^*}{C}, \frac{C}{C^*}\right) \leq \rho(n) \qquad \text{con } n = |x| \text{ e } x \in X$$

Ad esempio, se un algoritmo garantisce di restituire sempre soluzioni di costo al più il doppio (o almeno la metà) del costo della soluzione ottima, allora:

$$\max\left(\frac{C^*}{C}, \frac{C}{C^*}\right) \leq 2$$

e l'algoritmo si dice **2-approssimato** ($\rho(n) = 2$ in questo caso)

La distanza dall'ottimo può dipendere dalla dimensione n dell'istanza in input. Ad esempio, se

$$\max\left(\frac{C^*}{C}, \frac{C}{C^*}\right) \leq \log_2 n$$

allora la soluzione approssimata si allontana sempre di più dall'ottimo, al crescere della dimensione del problema. In questo caso, otteniamo una **$\log(n)$-approssimazione**.

ALGORITMO 2-APPROSSIMATO PER VERTEX COVER

Consideriamo il problema del **vertex cover** nella sua formulazione classica come problema di ottimizzazione.

Un **vertex cover** di un grafo $G = (V, E)$ è un sottinsieme $V' \subseteq V$ tale che, per ogni arco $(u, v) \in E$, almeno uno tra u e v appartiene a V'.

MIN-VERTEX-COVER: Dato un grafo non orientato G, restituire un **vertex cover** di **dimensione minima**.

Sappiamo che **MIN-VERTEX-COVER** è difficile almeno quanto **VERTEX-COVER**, che abbiamo già dimostrato essere **NP-COMPLETO**.

E', quindi, improbabile che possa essere risolto in modo esatto impiegando tempo polinomiale.

E', però, relativamente facile calcolare un **vertex cover** di dimensione non troppo superiore all'ottimo.

L'idea è quella di: selezionare di volta in volta un **arco** del grafo in modo casuale; aggiungere i suoi estremi a un insieme S inizialmente vuoto; **eliminare** dal grafo tutti gli **archi incidenti** nei 2 vertici appena aggiunti.

L'algoritmo risultante è il seguente:

```
APPROX-VERTEX-COVER (G)
    S = ∅
    E' = E
    A = ∅
    WHILE E' ≠ ∅ DO
        SCEGLI UN ARCO (v,u) ∈ E'
        A = A ∪ {(v,u)}
        S = S ∪ {v,u}
        E' = E' \ {(x,y) ∈ E | x=v o y=u}
    RETURN S
```

Le 2 istruzioni in rosso non sono necessarie ma utili alla dimostrazione

APPROX-VERTEX-COVER (G) restituisce un vertex cover di G in tempo polinomiale su $|G|$.

Dim. A ogni iterazione, l'algoritmo seleziona un arco $e \in E'$ e lo elimina da E' stesso. Quindi, il numero di iterazioni del ciclo WHILE è al più pari alla cardinalità iniziale di E', che è pari a $|E|$. L'operazione di rimozione degli archi da E' può chiaramente essere effettuata in tempo polinomiale su $|G|$ (con liste di adiacenza è possibile scrivere un algoritmo che impiega tempo lineare su $|G| = |V| + |E|$).

L'insieme S restituito è chiaramente un vertex cover, poiché contiene almeno un estremo per ogni arco di E. □

Se S^* è il minimo vertex cover di G e S è il vertex cover restituito da APPROX-VERTEX-COVER(G), allora

$$|S| \leq 2|S^*|$$

Dim. Consideriamo l'insieme degli archi che vengono selezionati ad ogni ripetizione del ciclo WHILE e inseriti nell'insieme A.
Osserviamo che gli archi contenuti in A non condividono tra loro nessuno degli estremi, cioè:

se $(u,v) \in A$ e $(u',v') \in A$, allora $\{u,v\} \cap \{u',v'\} = \emptyset$

Ciò è dovuto al fatto che ogni volta che un arco $e \in E$ viene inserito in A, tutti gli archi che condividono un estremo con e vengono eliminati da E' e non potranno essere inseriti in A successivamente.

Ora, se S^* è un vertex cover minimo di G, allora deve contenere almeno uno degli estremi di ciascun arco in A. Poiché tali archi non condividono estremi, S^* dovrà contenere almeno tanti vertici quanti sono gli archi di A

$$|S^*| \geq |A| \qquad (1)$$

L'insieme S costruito dall'algoritmo contiene 2 vertici per ogni arco in A, e tali vertici sono tutti distinti. Quindi:

$$|S| = 2 \cdot |A| \qquad (2)$$

Da (1) e (2) segue immediatamente che $|S| \leq 2|S^*|$ □

APPROX-VERTEX-COVER(G) è un algoritmo 2-approssimato per MIN-VERTEX-COVER.

ALGORITMO 2-APPROSSIMATO PER LO ZAINO 0/1

Sia dato un insieme di oggetti $I = \{1, 2, \dots, n\}$ e un numero naturale W. L'oggetto i-esimo ha peso w_i e valore v_i.
Assumiamo che I sia ordinato per valori decrescenti: $v_1 \geq v_2 \geq \dots \geq v_n$.
Sia (I, W) il problema di individuare il sottoinsieme di I che abbia peso $\leq W$ e valore massimo.

Denotiamo con $I_J^{0/1}$ il problema di individuare il sottoinsieme di I di peso $\leq W$, che contenga solo oggetti in $\{J, J+1, \dots, n\}$, e tale che:

1) l'oggetto J-esimo è contenuto nella soluzione
2) il valore complessivo della soluzione è massimo

Denotiamo con I_J^{frac} il problema di individuare un insieme di **Porzioni di oggetti** in I di peso complessivo $\leq W$, che contiene solo oggetti in $\{J, J+1, \dots, n\}$, e Tale che:

1) l'oggetto J-esimo è completamente contenuto nella soluzione

2) il **valore complessivo** delle porzioni degli oggetti contenuti della soluzione è massimo

$I_J^{0/1}$ e I_J^{frac} sono la versione 0/1 e la versione frazionaria dello stesso problema.

a) La soluzione ottima S^* all'istanza (I, W) del problema dello zaino 0/1 coincide con la soluzione ottima di uno dei problemi $I_J^{0/1}$, con $1 \leq j \leq n$. Cioè:

$$S^* \in \{P_1, P_2, \dots, P_n\}$$

dove P_J è la soluzione ottima al problema $I_J^{0/1}$.

Dim. Sia $S^* = J_1 J_2 \dots J_k$ soluzione ottima del problema (I, W). Assumiamo che la sequenza S^* sia ordinata in modo che $J_1 < J_2 < \dots < J_k$.
Chiaramente, S^* è anche soluzione ammissibile per il problema $I^{0/1}_{J_1}$, poiché contiene solo oggetti nell'insieme $\{J_1, J_1+1, \dots, n\}$ e l'oggetto J_1-esimo è contenuto in S^*. S^* è anche la soluzione ottima di $I^{0/1}_{J_1}$, altrimenti, se così non fosse, esisterebbe una soluzione S' che contiene J_1 e solo oggetti in $\{J_1, J_1+1, \dots, n\}$ di valore superiore. Ma S' sarebbe ammissibile per (I, W) perché ha peso non superiore a W. Questo contraddice l'assunzione di ottimalità di S^* per (I, W). □

b) La soluzione ottima Q_J del problema frazionario I^{fraz}_J è ottenibile:

- Selezionando l'oggetto J per intero
- aggiungendo il risultato dell'**algoritmo greedy** per lo **Zaino frazionario** con input $\{J+1, \dots, n\}$ e $W - w_J$.

Dim. La soluzione così ottenuta ha la forma

$$Q_J = (J, w_J)(J_1, q_1)\dots(J_k, q_k)$$

che deve soddisfare il vincolo di ammissibilità

$$\sum_{c=1}^{k} q_c \le W - w_J$$

del problema dello zaino frazionario classico con input $\{J+1, \dots, n\}$ e $W - w_J$.

Da questo otteniamo immediatamente che

$$w_J + \sum_{c=1}^{k} q_c \le W$$

che ci conferma che Q_J è ammissibile per I_J^{frac}. Poiché l'algoritmo greedy per lo zaino frazionario produce sempre l'ottimo per il problema che riceve in ingresso (vedi lezione ad esso dedicata), segue che Q_J è proprio la soluzione ottima per I_J^{frac}.

□

c) Per ogni $1 \le j \le m$, Q_j contiene al più una sola componente frazionaria di qualche oggetto (tutte le altre componenti sono oggetti interi). Cioè, ogni elemento in Q_j è del tipo (j, w_j), tranne al più l'ultimo che è del tipo (k, q_k) con $q_k \le w_k$.

Dim. Q_j ha la forma $(j, w_j)(j_1, q_1) \dots (j_k, q_k)$ per il vincolo di ammissibilità del problema Γ_j^{frac}. Inoltre sappiamo che $(j_1, q_1) \dots (j_k, q_k)$ è stata ottenuta tramite l'algoritmo greedy per lo zaino frazionario con input $(\{j+1, \dots, n\}, W - w_j)$. La prima scelta greedy è quella di inserire la massima quantità possibile dell'oggetto col massimo rapporto $\frac{v}{w}$ (valore su peso).
Sia tale oggetto proprio j_1, allora abbiamo 2 casi:

1) $w_{j_1} > W - w_j$: in tal caso, $q_1 = W - w_j$ e $k = 1$, poiché non vi è più spazio nello zaino che, alla fine, conterrà solo j e j_1. Poiché j è interamente contenuto, la tesi è verificata.

2) $w_{J_1} \leq W - w_J$: in tal caso, $q_1 = w_{J_1}$. Quindi, sia J che J_1 sono interamente contenuti nella soluzione, inoltre, $(J_2, q_2) \dots (J_k, q_k)$ è soluzione ottima del sottoproblema frazionario $(\{J_1+1, \dots, n\}, W - (w_J + w_{J_1}))$.
Ripetendo lo stesso ragionamento per tutte le scelte greedy contenute in Q_J, si troveremo o sempre nel caso 2) oppure nel caso 2) fino alla $(k-1)$-esima scelta e nel caso 1) alla k-esima scelta.
Di conseguenza, tutti gli oggetti selezionati saranno presi interamente, tranne al più l'ultimo. □

d) Per ogni $1 \leq J \leq n$, sia $\hat{Q}_J$ la soluzione ottenuta eliminando da Q_J la porzione ma intera.
Allora, $\hat{Q}_J$ è una soluzione ammissibile del problema $I_J^{0/1}$ e

$$C(\hat{Q}_J) \geq \frac{C(P_J)}{2}$$

Dim. Ogni soluzione per un problema dello zaino 0/1 è ammissibile per la corrispondente istanza dello zaino frazionario (poiché quest'ultima ha meno vincoli).
Quindi, vale certamente che

$$C(Q_J) \geq C(P_J) \qquad (1)$$

per ogni $1 \leq J \leq m$.
Preso un arbitrario $1 \leq J \leq m$, per la proprietà c) sappiamo che

$$Q_J = (J, w_J)(J_1, w_{J_1}) \dots (J_{k-1}, w_{J_{k-1}})(J_k, q_k)$$

con $q_k \leq w_{J_k}$. Allora

$$C(Q_J) = v_J + v_{J_1} + \dots + v_{J_{k-1}} + q_k \cdot \frac{v_{J_k}}{w_{J_k}} \qquad (2)$$

Poiché gli oggetti sono ordinati per valore non crescente, vale che $v_J \geq v_{J_k}$. Inoltre, poiché $q_k \leq w_{J_k}$, vale che $v_{J_k} \geq q_k \cdot \frac{v_{J_k}}{w_{J_k}}$, da cui otteniamo che

$$v_J \geq q_k \cdot \frac{v_{J_k}}{w_{J_k}}$$

Ma allora vale anche che

$$v_J + v_{J_1} + \dots + v_{J_{k-1}} \geq q_k \cdot \frac{v_{J_k}}{w_{J_k}} \qquad (3)$$

e sommando $(v_J + v_{J_1} + \dots + v_{J_{k-1}})$ a entrambi i membri della disuguaglianza (3), otteniamo che

$$2 \cdot (v_J + v_{J_1} + \dots + v_{J_{k-1}}) \geq (v_J + v_{J_1} + \dots + v_{J_{k-1}}) + q_k \cdot \frac{v_{J_k}}{w_{J_k}} \qquad (4)$$

Per definizione, $\hat{Q}_J = (J, w_J)(J_1, w_{J_1}) \dots (J_{k-1}, w_{J_{k-1}})$ e quindi

$$C(\hat{Q}_J) = (v_J + v_{J_1} + \dots + v_{J_{k-1}}) \qquad (5)$$

Da (2), (4) e (5) otteniamo che

$$2 \cdot C(\hat{Q}_J) \geq C(Q_J)$$

e dalla (1), per transitività, si ha

$$2 \cdot C(\hat{Q}_J) \geq C(P_J)$$

da cui discende immediatamente la tesi. □

ALGORITMO APPROSSIMATO

Consideriamo l'algoritmo $A(I, W)$ che, per ogni $1 \leq J \leq m$:

1) Calcola la soluzione Q_J utilizzando l'algoritmo greedy per lo zaino frazionario;
2) Ottiene $\hat{Q}_J$ da Q_J eliminando l'eventuale oggetto non intero;
3) restituisce il massimo dei Q_J calcolati.

TEOREMA: Se $S = A(I, W)$ e S^* è la soluzione ottima del problema dello zaino 0/1 con input (I, W), allora

$$C(S) \geq \frac{C(S^*)}{2}$$

Dim. Dalla proprietà d) sappiamo che $C(\hat{Q}_J) \geq \frac{C(P_J)}{2}$ per ogni $1 \leq J \leq m$.

Dalla proprietà a) sappiamo che S^* è uguale a uno dei P_J.

Infine, S è uguale al più grande dei $\hat{Q}_J$.

Mettendo tutto insieme, otteniamo proprio la Tesi. □

Quindi, $A(I, W)$ è un algoritmo 2-approssimato per il problema dello Zaino 0/1.

E' anche immediato che il tempo richiesto da $A(I, W)$ è polinomiale su $n = |I|$, in particolare $O(n^2)$, quindi indipendente da W.